

ST-AGS: Space-Time Anchored 4D Gaussian Splatting for Geometrically-Aligned Dynamic Scene Reconstruction

Tianjiao Jing, Chang Liu, Ziqi Xi, Shi-Sheng Huang, Jianwei Guo, and Hongbo Fu

Abstract—Dynamic scene reconstruction with 4D Gaussian Splatting has achieved much progress in dynamic rendering quality and speed, but still faces bottlenecks in huge storage overhead. Recent anchor-based Gaussian splatting show great potential to reduce 4D Gaussian primitives, however, their spatiotemporal *compactness* remain unexplored. This paper proposes a new anchor-based 4D Gaussian splatting, i.e., ST-AGS, which significantly improves the spatiotemporal *compactness* of 4D anchor representation for much less storage. Our first key innovation is the introduce of *Space-Time Anchors* (ST-Anchors), in which we explicitly model the spatiotemporal motion of 4D anchors using controllable spatiotemporal curves. Based on the ST-Anchors, we propose a Motion-Aware ST-Anchor Allocation strategy to robustly grows anchors, and further investigate a geometrically-aligned 4D optimization for geometrically *compact* 4D scene reconstruction from videos. By extensive evaluation on public datasets, our ST-AGS costs about average $2.78\times$ fewer storage consumption than the latest representative anchor-based 4DGS, while achieving better dynamic rendering quality with best performance for the balance between rendering quality and memory storage benefit from the *compact* 4D reconstruction. Besides, our ST-AGS also enables controllable motion editing application by explicitly manipulating the ST-Anchors, achieving state-of-the-art rendering quality and storage consumption as a new anchor-based 4D Gaussian splatting.

Index Terms—4D Gaussian Splatting, Space-Time 4D Anchors, Geometrically-Aligned Reconstruction

I. INTRODUCTION

ACCURATE dynamic reconstruction from videos continues to be a challenging problem [1]–[3], which serves as foundation for many applications like dynamic view synthesis [4]–[8], digital avatars [9]–[14], and teleconference [15]–[17]. Unlike traditional depth-based warping-and-inpainting approaches [2], [18], the success of NeRF [19] with extensive extension to dynamic scenarios [6], [20] have achieved remarkable dynamic view synthesis results. Meanwhile, due to the huge computation consumption, their practical applications are still limited in computer graphics community even though impressive acceleration mechanisms [21]–[24] proposed.

Tianjiao Jing, Ziqi Xi, Shi-Sheng Huang, and Jianwei Guo are with the School of Artificial Intelligence, Beijing Normal University, Beijing, China. E-mail: {202621081020, 202411081099}@mail.bnu.edu.cn; {huangss, jianwei.guo}@bnu.edu.cn. Corresponding author: Shi-Sheng Huang.

Chang Liu is with the School of Information, Renmin University of China, Beijing, China. E-mail: liuchang1122@ruc.edu.cn.

Hongbo Fu is with the Division of Arts and Machine Creativity, The Hong Kong University of Science and Technology, Hong Kong, China. E-mail: hongbofu@ust.hk.

The pioneering work of 3D Gaussian Splatting [25], [26] inspired many subsequent works for dynamic scene reconstruction using 4D Gaussian splatting [27], which have achieved impressive dynamic view synthesis results at very efficient rendering speed. The current 4DGS approaches can be mainly divided into two types including deformation field based 4DGS [28]–[31] and explicitly time-dependent 4DGS [32]–[38], where the former types adopt a MLP-based deformation networks to predict dynamic Gaussian primitives and the latter model the Gaussian motions as time-dependent function. However, one main drawback of most current 4DGS approaches comes from the significant memory storage consumption due to the redundant dynamic Gaussian primitives densification, which would easily cost above GB level GPU memory even for short videos. To effectively reduce the memory storage, some subsequent works adopt to prune the Gaussian primitives using spatiotemporal cues [39] or vector quantization based Gaussian compression [40], which successfully compress the storage to tens of MB level. However, they often face quality degradation with poorer dynamic view synthesis rendering results.

Recently, some approaches proposed to use anchor-based neural Gaussian splatting [41], [42] to dynamic view synthesis [43]–[45], where the dynamic scenes are first compressed into anchor points and then decoded as Gaussian primitives using pre-trained MLP networks. In this way, the memory storage can be effectively reduced from Gaussian primitives to much less anchor points, while preserving the dynamic reconstruction quality as much as possible. Nevertheless, their dynamic anchor assignment mechanisms still suffer from severe temporal redundancy, leaving further room for GPU memory compression. More importantly, the dynamic Gaussian primitives reconstructed by these methods often lack of geometric compactness. When facing highly dynamic scenes with intensive motions [46], [47], their reconstructed dynamic geometric surfaces would easily lead to severe degradation of dynamic reconstruction quality. Besides, they also lack the capability of controllable dynamic motion editing, such as applications like reanimating [48], [49].

In this paper, we propose a new anchor-based 4D Gaussian splatting approach i.e., ST-AGS, which significantly reduces the dynamic Gaussian primitives storage while achieving high fidelity dynamic rendering quality. Unlike redundantly allocating anchors in 4D space [43]–[45], we introduce a new ST-Anchors representation, which explicitly model the anchors’ dynamic motion using controllable curves thus effectively

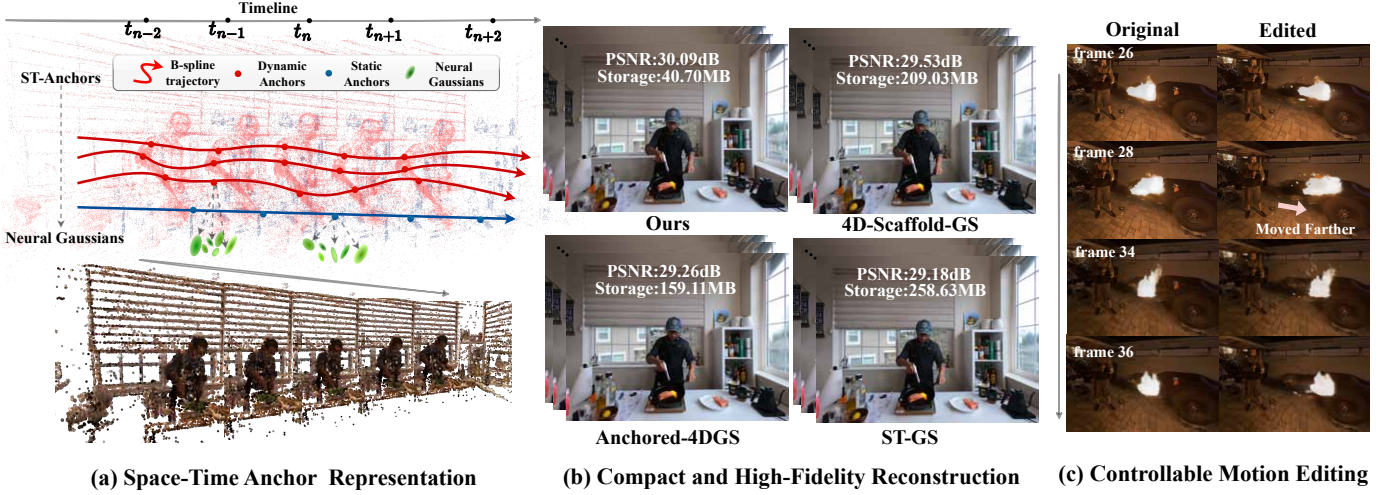


Fig. 1. **ST-AGS enables compact and high-fidelity dynamic scene reconstruction.** By introducing persistent space-time anchors with explicit B-spline trajectories, ST-AGS captures dynamic scene evolution with a compact 4D representation. (a) The proposed representation explicitly models temporal motion at the anchor level and decodes local neural Gaussians for high-quality rendering. (b) ST-AGS achieves superior rendering quality with substantially lower storage requirements than 4DGS baselines. (c) ST-AGS also supports controllable motion editing by manipulating anchor trajectories. Additional results are provided in the supplemental video.

alleviating temporal redundancy. A key challenging lies in properly performing densification for ST-Anchors, to retain their temporal-dependency benefit while enabling high fidelity dynamic scene reconstruction. To overcome this issue, we propose a motion-aware ST-Anchors allocation mechanism to adaptively grow 4D anchors in fast motion region in a robust manner. Moreover, we further build up a geometrically-aligned ST-Anchors optimization to further improve the compactness of Gaussian primitives for geometrically compact 4D scene reconstruction from videos. Besides, ST-Anchors’s motion representation using controllable curves can also bring useful cue to perform controllable motion editing, by directly manipulating the control curves, which is not feasible for previous works [37], [38], [43]–[45].

To verify the effectiveness of the proposed method, we conduct comprehensive evaluations on three public dynamic-scene datasets, including N3DV [50], Google Immersive [51], and ENeRF-Outdoor [52]. Qualitative and quantitative comparisons are performed against similar anchor-based 4DGS (including 4D-Scaffold-GS [45], Anchored-4DGS [44] and ADC-GS [53]) and baseline 4DGS approaches (including 4DGaussian [27], ST-GS [35], and 4D-Rotor-GS [34]). Compared with the recent anchor-based 4DGS, ST-AGS achieves an average PSNR improvement of 0.63dB than state-of-the-art representative 4D-Scaffold-GS [45] across the three datasets, while using about $2.78\times$ fewer memory storage (reducing the average model storage from 138.85 MB to 50.39 MB) with even more storage reduction than Anchored-4DGS [44]. Moreover, ST-AGS requires only about 13%–19% as many anchors as 4D-Scaffold-GS, demonstrating substantially improved spatiotemporal representation compactness without sacrificing reconstruction quality. ST-AGS achieves slightly more memory storage than ADC-GS, but achieves much better dynamic rendering quality. Similarly, ST-AGS consistently outperforms these 4DGS baselines, which shows

that our ST-AGS tracks the best performance for the balance between dynamic rendering quality and memory storage. Besides, ST-Anchor trajectories naturally support trajectory-based controllable motion editing through direct manipulation of the learned anchor motion, which is not feasible by these previous baselines.

II. RELATED WORK

Dynamic scene reconstruction from videos aims to reconstruct the temporal varying geometry contents of 3D scenes, which has long been a challenging research problem in the fields of computer graphics and computer vision [54], [55]. Here we select most relevant fields for discussion inducing dynamic view synthesis, dynamic surface reconstruction and Gaussian splatting compression.

Dynamic View Synthesis. Dynamic view synthesis aims to generate novel view of dynamic 3D scene from multi-view videos, which has received much research attention since the pioneering work of depth based image rendering [2], [18], [56]. Subsequent works further improved the rendering quality [57], efficiency [58] and spatiotemporal consistency [8], [59], however, the dynamic view synthesis quality is still limited by the depth estimation and inpainting quality. NeRF [19] and its extension to dynamic view synthesis with dynamic representation decoupling [60], [61], deformation fields [62], [63] and incremental reusing [64], [65]. Although later works impressively accelerate NeRF with neural primitives [21], [23] and further improve the dynamic rendering using lightweight architecture such as MPI [66], HexPlane [67] and hybrid neural fields [68], there still exists inevitable rendering latency due to the volumetric rendering for NeRF. Recently, the success of 3DGS [25] quickly inspired many 4D Gaussian splatting works [27]–[31] with more better of spatiotemporal consistency [28], rendering efficiency [27], [28] and scalability such as weakly textured scenes [30]. Another kinds of

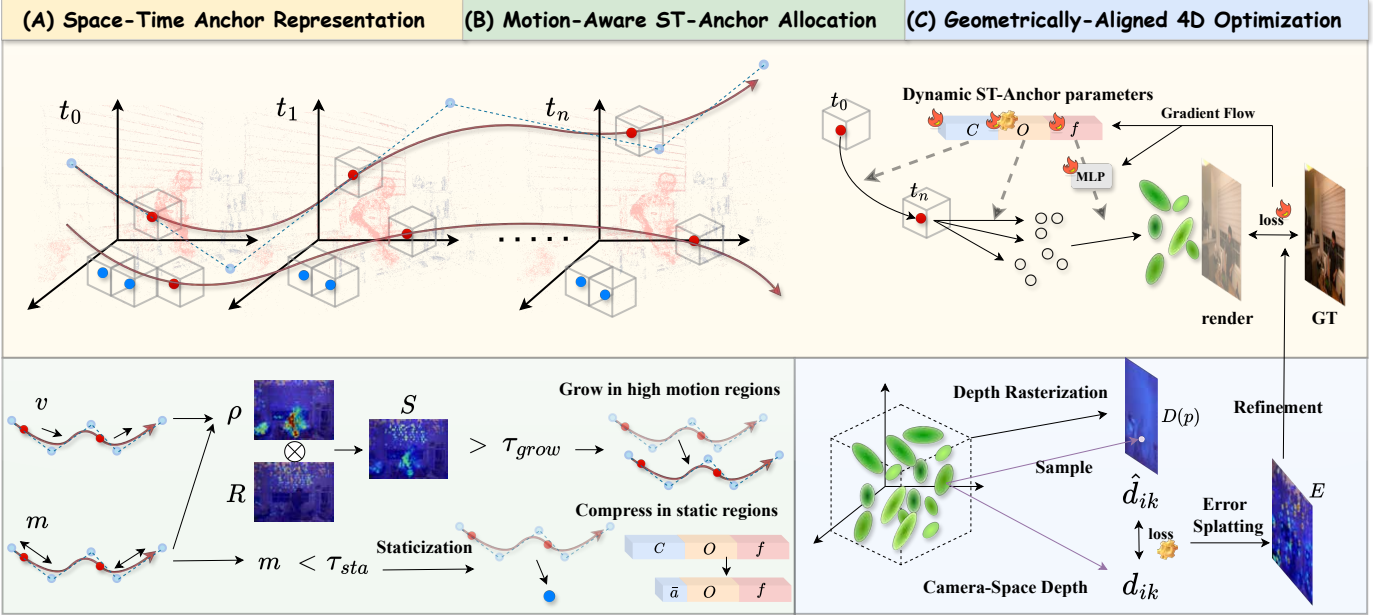


Fig. 2. **Overview of the proposed ST-AGS framework.** (A) We represent dynamic scenes with Space-Time Anchors (ST-Anchors), whose motion is explicitly modeled by B-spline trajectories and decoded into local neural Gaussians for rendering. (B) Based on the explicit anchor trajectories, our motion-aware allocation adaptively grows anchors in motion-intensive regions while staticizing near-stationary anchors to remove redundant temporal parameters. (C) We further introduce a geometrically-aligned 4D optimization, where the rendered depth provides self-induced geometric guidance for local Gaussian alignment and depth-guided photometric refinement. Together, these components enable compact and high-fidelity dynamic scene reconstruction.

4DGS adopt to explicit motion modeling [32]–[38] with more compact 4D scene representation. Since they aim to characterize dynamic motion as comprehensively as possible, these methods typically involve an excessive number of Gaussian primitives, resulting in very high GPU memory consumption.

Very recently, some approaches introduce anchor-based neural Gaussians [41] for more rational 4D Gaussian distribution [43]–[45]. But most of them adopt to spatially allocate 4D anchors, there remains further potential to compress the redundancy of anchor generation and representation. In contrast, this paper introduces a novel ST-Anchors representation, explicitly modeling the motion of 4D anchors to further compress the temporal redundancy thus significantly reducing memory usage, and ensures high-quality dynamic view synthesis through a compact ST-Anchors optimization mechanism.

Dynamic Scene Reconstruction. The traditional dynamic scene reconstruction [69], [70] uses depth sensors to build a TSDF representation for dynamic geometry and appearance modeling. The subsequent works further refined the dynamic reconstruction results [71] and robustness of non-rigid tracking for dynamic humans [72], [73]. NeRF-based approaches [20], [74], [75] adopt to learn dynamic implicit fields for better dynamic geometry reconstruction. Recently, many 3DGS based dynamic surface reconstruction have been proposed by joint optimizing Gaussian primitives with mesh templates [76], [77], SDF-based implicit fields [78], [79] and SMPL-X [80] based human avatar reconstruction [9]–[14]. Different from those previous works, our approach provides a compact and geometrically-aligned 4DGS optimization based on a new space-time 4D anchors, which achieves more better dynamic

scene reconstruction while enabling dynamic reanimating towards expansive 4D video content editing [6], [48], [49], [81].

Gaussian Splatting Compression. Gaussian primitive pruning and attribution compression has been an effective solution for the bottleneck of substantial memory costs in static 3DGS, with impressive Gaussian splatting compression approaches using locality-aware compact representation [82], learnable mask [83] and vector quantization (VQ) based compression [39], [40]. For dynamic Gaussian splatting requiring significant number of Gaussian primitives, lightweight 4DGS compression approaches adopt to perform spatiotemporal Gaussian pruning [29], deformable 3DGS compression [53] and motion decoupling [37]. However, this kind of Gaussian splatting compression would lead to quality degradation. In difference, our approach proposes to reduce the temporal redundancy of 4D anchors based on the neural Gaussian splatting framework [41], which can effectively decrease the memory storage while preserving the dynamic reconstruction quality.

III. ST-AGS

Given a set of synchronized multi-view videos of a dynamic scene, our goal is to reconstruct a compact 4D Gaussian representation that supports high-quality novel-view rendering while substantially reducing the storage overhead introduced by temporal modeling. Unlike previous anchor-based 4DGS [44], [45], we propose **ST-AGS**, a space-time anchored Gaussian representation that models scene dynamics at the anchor level. As illustrated in Fig. 2, our method consists of three main components. First, we introduce **Space-Time**

Anchors (ST-Anchors), where each dynamic anchor is associated with an explicit cubic B-spline trajectory shared by its local neural Gaussians. Second, we develop a **Motion-Aware ST-Anchor Allocation** strategy that grows additional anchors in regions requiring greater spatiotemporal capacity while staticizing near-static anchors to remove redundant temporal parameters. Finally, we introduce a **Geometrically-Aligned 4D Optimization** that improves the spatial organization of local neural Gaussians and preserves reconstruction fidelity under the compact representation.

A. Space-Time Anchor Representation

One major issue for previous anchor-based 4DGS [44], [45] is that they simply allocate anchors in 4D space without considering the temporal redundancy between allocated anchors, generating a considerable number of redundant anchors with substantial memory overhead.

Instead, we extend simple anchor representation into **Space-Time Anchors (ST-Anchors)**, and explicitly model scene dynamics directly at the anchor level. Neural Gaussians generated from the same anchor typically describe correlated local content and therefore need not maintain independent motion trajectories. We refer to the learnable parameters associated with a dynamic ST-Anchor as its **Dynamic ST-Anchor Parameters**, including the B-spline control points $\mathbf{C}_i = \{\mathbf{c}_{i,j}\}_{j=0}^{M-1}$, the anchor feature $\mathbf{f}_i$, local offsets $\mathbf{O}_i = \{\mathbf{o}_{ik}\}_{k=1}^K$, and additional spatial attributes. Among them, $\mathbf{C}_i$ governs the temporal motion of the anchor, $\mathbf{O}_i$ defines the local spatial arrangement of neural Gaussians, and $\mathbf{f}_i$ provides the shared feature for Gaussian attribute decoding.

Specifically, given a normalized timestamp $t \in [0, 1]$, let s denote the temporal segment containing t and $u \in [0, 1]$ its local coordinate. The position of the i -th ST-Anchor is defined as

$$\mathbf{a}_i(t) = \sum_{\ell=0}^3 B_\ell(u) \mathbf{c}_{i,s+\ell}, \quad (1)$$

where $B_\ell(\cdot)$ denotes the cubic B-spline basis function. Cubic B-splines provide local support, such that each trajectory position depends only on four neighboring control points. This locally supported parameterization allows flexible adjustment of complex local motion without coupling trajectory parameters over the entire temporal domain.

At each timestamp, the B-spline trajectory first moves the persistent ST-Anchor to $\mathbf{a}_i(t)$. The learned local offsets then instantiate K neural Gaussian centers around the moved anchor. Specifically, the center of the k -th neural Gaussian is given by

$$\mathbf{x}_{ik}(t) = \mathbf{a}_i(t) + \mathbf{o}_{ik}, \quad (2)$$

where $\mathbf{o}_{ik}$ denotes the learned local offset of the k -th neural Gaussian. The B-spline trajectory captures the shared coarse motion of the local Gaussian group, while the local offsets preserve its fine spatial structure. Sharing one trajectory among multiple local neural Gaussians avoids repeatedly encoding correlated motion and forms the basis of the temporal compactness of ST-AGS.

After determining the Gaussian centers, the shared anchor feature $\mathbf{f}_i$, together with the viewing direction and temporal encoding, is fed into the neural Gaussian decoders to predict color, opacity, and covariance for the generated Gaussians. The resulting neural Gaussians are then rasterized for image reconstruction, as illustrated in Fig. 2. In this way, the B-spline control points govern temporal motion, the local offsets determine fine spatial structure, and the time-conditioned decoder predicts the remaining rendering attributes.

B. Motion-Aware ST-Anchor Allocation

The explicit trajectories of ST-Anchors provide a natural cue for estimating the temporal complexity of different scene regions. Conventional anchor densification mainly relies on reconstruction gradients to determine where additional spatial capacity is required. However, rapidly moving content may only appear in a subset of frames and therefore accumulate weaker reconstruction evidence than persistently visible regions, despite requiring greater spatiotemporal modeling capacity. To make the ST-Anchors accurately coherent with the dynamic motion, we therefore propose a motion-aware ST-Anchors allocation with adaptive growing and staticization for robust ST-Anchors allocation.

1) *Motion-Aware ST-Anchor Growing*: For each growing candidate, we denote its reconstruction demand by R , computed as the average magnitude of the training-loss gradient with respect to its projected 2D Gaussian center over valid observations. For its parent ST-Anchor, we characterize temporal complexity using two complementary trajectory cues: the trajectory magnitude m , obtained by accumulating the displacement between neighboring B-spline control points, and the motion rate v , measured from the average derivative magnitude of the continuous trajectory. After normalizing m with a robust scene-level motion reference to obtain $\hat{m}$, we define the motion-aware capacity-demand prior as

$$\rho = (1 + \lambda_m \hat{m}) e^{\gamma v}. \quad (3)$$

A larger ρ indicates a stronger demand for spatiotemporal representation capacity. We then combine the reconstruction demand R with the motion-aware prior ρ to define the final growing score

$$S = \rho R, \quad S > \tau_{\text{grow}}. \quad (4)$$

Here, the reconstruction demand R indicates *where* the current representation is insufficient, while the motion-aware prior ρ determines *how strongly* additional spatiotemporal capacity is required. Their combination forms the growing score S , which preferentially allocates new ST-Anchors to regions with both high reconstruction demand and complex motion, as illustrated in Fig. 2.

A newly generated ST-Anchor requires not only an appropriate spatial location but also a meaningful temporal trajectory. For each selected growing candidate, we determine a representative timestamp from its reconstruction evidence and locate the child at the corresponding local Gaussian position. Rather than initializing the child with a stationary trajectory, we translate all control points of its parent by a common spatial

offset. This preserves the learned trajectory shape while moving the child to the newly allocated spatial location, providing a motion-consistent initialization for subsequent optimization.

2) *Motion-Aware ST-Anchor Staticization*: While motion-aware growing allocates additional representation capacity to regions with complex motion, the opposite operation is equally important for compact 4D reconstruction. Dynamic scenes typically contain large static backgrounds and slowly varying structures whose learned trajectories remain nearly stationary. Maintaining complete B-spline trajectories for these anchors introduces temporal parameters that contribute little to dynamic scene modeling.

After the ST-Anchor trajectories have been sufficiently optimized, we identify near-static anchors whose trajectory magnitude m falls below a spatial-resolution-aware threshold τ_{sta} . For each such anchor, we replace its time-varying B-spline trajectory with the temporal centroid $\bar{\mathbf{a}}$ of its sampled trajectory positions and remove the corresponding B-spline control points. As illustrated in Fig. 2, this converts the **Dynamic ST-Anchor Parameters** into a more compact set of **Static ST-Anchor Parameters**: the trajectory parameters $\mathbf{C}$ are replaced by a single static position $\bar{\mathbf{a}}$, while the local offsets $\mathbf{O}$, anchor feature $\mathbf{f}$, and spatial attributes are retained for neural Gaussian generation.

Importantly, staticization only removes redundant geometric temporal degrees of freedom and does not force the appearance of an anchor to remain constant over time. The timestamp is still provided to the neural Gaussian decoder, allowing color, opacity, and covariance to remain time-dependent even when the anchor position is fixed. This separation enables geometrically static regions to retain temporal appearance variations caused by illumination, shadows, or occlusions.

Together, motion-aware growing and staticization constitute the **Motion-Aware ST-Anchor Allocation** illustrated in Fig. 2. Complex dynamic regions receive additional anchors, while near-static regions discard unnecessary trajectory parameters, concentrating temporal degrees of freedom on regions where motion actually requires them.

C. Geometrically-Aligned 4D Optimization

The ST-Anchor representation and motion-aware allocation substantially reduce redundant temporal capacity. However, a compact parameterization alone does not guarantee that the remaining local Gaussians are spatially well organized. Under photometric supervision, multiple Gaussians may explain similar image observations while being dispersed along the viewing direction, which becomes particularly problematic when the representation budget is restricted.

We therefore introduce a **Geometrically-Aligned 4D Optimization** that derives geometric guidance directly from the current Gaussian representation without relying on external depth sensors or pretrained depth estimators. As illustrated in Fig. 2, we first establish Gaussian depth consistency and then propagate the resulting geometric discrepancy to image space through **Error Splatting** for geometry-guided refinement.

Gaussian depth consistency. We first perform **Depth Rasterization** by alpha-compositing the camera-space depths of the current neural Gaussians:

$$D(\mathbf{p}) = \sum_j T_j(\mathbf{p}) \alpha_j(\mathbf{p}) d_j, \quad (5)$$

where d_j denotes the camera-space depth of the j -th Gaussian and $T_j(\mathbf{p})$ is the accumulated transmittance before it. The resulting depth map $D(\mathbf{p})$ provides a view-dependent geometric consistency reference induced by the current Gaussian representation.

For each valid local Gaussian, we denote its **Camera-Space Depth** as d_{ik} and its projected image location as $\mathbf{p}_{ik}$. We sample the detached depth map at $\mathbf{p}_{ik}$ to obtain the consistency reference $\hat{d}_{ik}$ and minimize their relative discrepancy:

$$\mathcal{L}_{\text{align}} = \frac{1}{|\mathcal{V}|} \sum_{(i,k) \in \mathcal{V}} \frac{|d_{ik} - \hat{d}_{ik}|}{\hat{d}_{ik} + \epsilon}, \quad (6)$$

where $\mathcal{V}$ denotes the set of valid local Gaussians. Invalid or heavily occluded projections are excluded from the consistency constraint. During this optimization, both the rendered depth reference and the ST-Anchor trajectory are detached, so $\mathcal{L}_{\text{align}}$ primarily refines the anchor-relative offsets and spatial scales of local Gaussians without disturbing the learned B-spline motion.

Geometry-guided refinement. The same relative depth discrepancy provides a Gaussian-level measure of geometric inconsistency. We project these discrepancies back to the image plane and aggregate them according to their image-space contributions, producing a geometry inconsistency map $\mathbf{E}$. We refer to this operation as **Error Splatting**, as shown in Fig. 2, which converts local Gaussian geometric errors into an image-space signal for subsequent refinement.

Based on $\mathbf{E}$, we construct a geometry-guided weight map $\mathbf{W}_d = \mathbf{1} + \beta \mathbf{E}$ and use it to emphasize photometric errors in geometrically inconsistent regions:

$$\mathcal{L}_{\text{dRGB}} = \frac{\|\mathbf{W}_d \odot (\hat{\mathbf{I}} - \mathbf{I})\|_1}{\|\mathbf{W}_d\|_1}, \quad (7)$$

where $\hat{\mathbf{I}}$ and $\mathbf{I}$ denote the rendered and input images, respectively. Regions with larger geometric inconsistency therefore receive stronger photometric supervision, encouraging the compact representation to devote more optimization effort to geometrically challenging regions.

The overall training objective combines the standard photometric reconstruction loss with the proposed geometric terms:

$$\mathcal{L} = (1 - \lambda_{\text{ssim}}) \mathcal{L}_1 + \lambda_{\text{ssim}} \mathcal{L}_{\text{SSIM}} + \lambda_{\text{align}} \mathcal{L}_{\text{align}} + \lambda_{\text{dRGB}} \mathcal{L}_{\text{dRGB}}, \quad (8)$$

where $\mathcal{L}_{\text{SSIM}} = 1 - \text{SSIM}$. The standard photometric terms preserve appearance fidelity, $\mathcal{L}_{\text{align}}$ improves the spatial organization of local neural Gaussians, and $\mathcal{L}_{\text{dRGB}}$ focuses reconstruction on regions with stronger geometric inconsistency.

Together, the three components of ST-AGS form a compact and effective 4D representation: Space-Time Anchors reduce redundant temporal parameterization, Motion-Aware

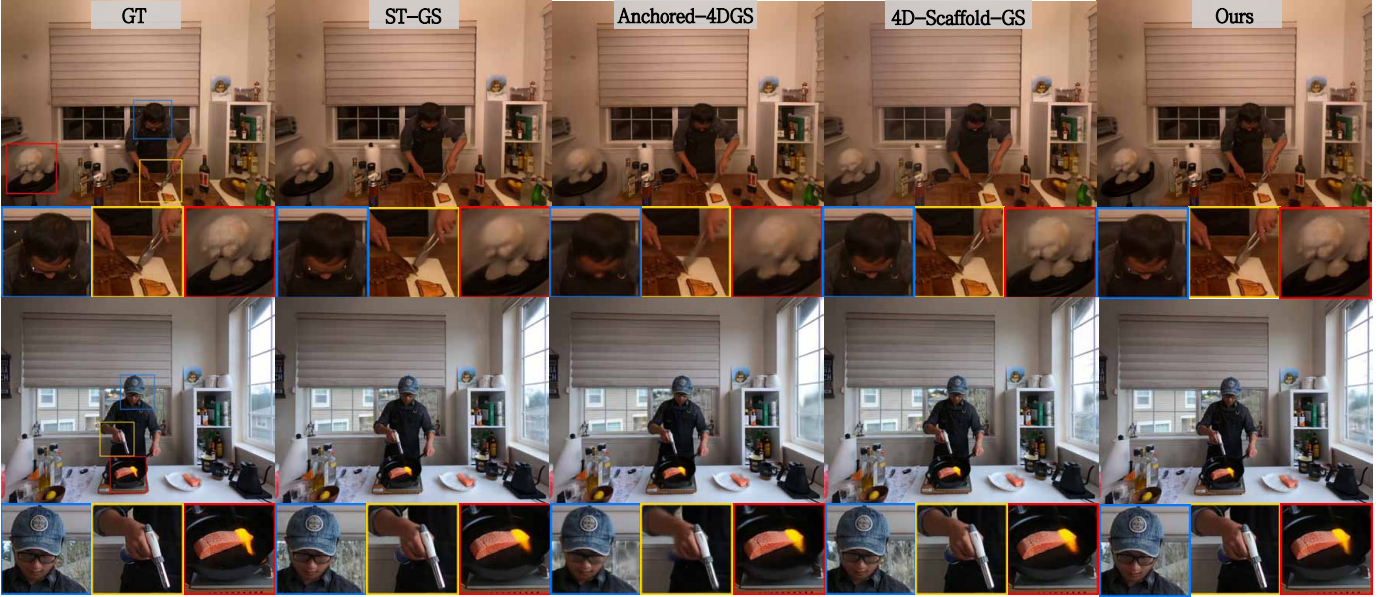


Fig. 3. Visual comparison results evaluated on N3DV dataset from different comparing approaches including ST-GS [35], Anchored-4DGS [44], 4D-Scaffold-GS [45] and ours respectively.

ST-Anchor Allocation concentrates temporal capacity according to local motion complexity, and Geometrically-Aligned 4D Optimization improves reconstruction fidelity under the compact representation.

Static and Dynamic Anchor Identification. To make more accurate learning of ST-Anchors, we perform static and dynamic anchor identification during the optimization. Specifically, all anchors are initially represented as dynamic ST-Anchors with B-spline trajectories. As optimization proceeds, anchors whose learned trajectory magnitude remains below the staticization threshold are identified as static and converted into static ST-Anchors by removing their redundant trajectory parameters, while anchors with significant motion retain the dynamic ST-Anchor representation.

IV. EXPERIMENTS

A. Datasets

We evaluate our method on three real-world datasets: Neural 3D Video dataset [50] (N3DV), Google Immersive dataset [51], and ENeRF-Outdoor dataset [52].

Neural 3D Video dataset (N3DV) comprises six real-world dynamic scenes: *coffee martini*, *cook spinach*, *cut roasted beef*, *flame salmon*, *flame steak*, and *sear steak*. We train and evaluate our method on all six scenes. Following previous works, we downsample the images to 1352×1014 and use cam0 as the test viewpoint.

Google Immersive dataset contains indoor and outdoor dynamic scenes captured by a 46-camera rig. We use the undistorted images and further downsample them to 1280×960 . We evaluate on four scenes: *02 Flame*, *09 Exhibit*, *10 Face Paint 1*, and *12 Cave*, while holding out the center camera as the test viewpoint.

Ennerf-Outdoor dataset contains outdoor dynamic scenes captured by 18 synchronized cameras. We downsample the

images to 960×540 and evaluate on the *actor1-4* and *actor2-3* sequences using the first 300 frames.

B. Implementation Details

Initialization. For all datasets, we initialize ST-AGS from the sparse point cloud reconstructed by COLMAP and voxelize the input points to obtain the initial ST-Anchor set. For dynamic sequences, we use only the sparse point cloud corresponding to the first frame of the selected temporal range, avoiding the mixture of dynamic geometry from different timestamps in the initial representation. All B-spline control points are initialized at the initial ST-Anchor position, so each anchor starts with a stationary trajectory. During training, the trajectory is gradually optimized from multi-view supervision to capture the motion of the scene. Each ST-Anchor generates $K = 10$ neural Gaussians and uses a cubic B-spline with $M = 48$ control points to model its temporal trajectory. All B-spline control points are initialized with the initial spatial position of the corresponding ST-Anchor. The remaining anchor attributes and neural Gaussian decoders are initialized following Scaffold-GS [41].

Training. We optimize ST-AGS end-to-end using the Adam optimizer. For all dynamic sequences, timestamps are linearly normalized to $[0, 1]$ according to the frame index. During training, we randomly sample one observation from all training timestamp-view pairs at each iteration. To reduce the memory consumption of long sequences, images are loaded on demand, while only camera parameters and image indices are maintained in memory instead of preloading all video frames. After an initial optimization stage, motion-aware ST-Anchor growing is periodically performed to adaptively allocate representation capacity according to reconstruction demand and the motion information provided by the explicit trajectories. Once the ST-Anchor trajectories have been suf-



Fig. 4. Visual comparison results evaluated on Google Immersive dataset from different comparing approaches including ST-GS [35], Anchored-4DGS [44], 4D-Scaffold-GS [45] and ours respectively.

ficiently optimized, we further perform motion-aware staticization to remove redundant B-spline trajectory parameters from near-static anchors. The training objective consists of the standard photometric reconstruction loss, the geometric alignment loss, and the geometry-guided photometric loss. All experiments are conducted on a single NVIDIA GeForce RTX 5090 GPU.

C. Baselines and Evaluation metrics

We compare ST-AGS with representative dynamic Gaussian splatting methods, including both primitive-level dynamic representations and recent anchor-based approaches. For primitive-level methods, we include 4DGaussian [27], ST-GS [35], and 4D-Rotor-GS [34], which model scene dynamics by introducing temporal motion or deformation to Gaussian primitives. We further compare with recent anchor-based methods, including ADC-GS [53], Anchored-4DGS [44], and 4D-Scaffold-GS [45], which organize dynamic scene representations around shared anchors and provide strong baselines for evaluating representation efficiency.

For methods with publicly available official implementations, we use the official code and follow the recommended settings for each dataset. For ST-GS, we report two configurations following its segment-based training strategy: ST-GS^{/50} divides each sequence into 50-frame segments, while ST-GS^{/100} uses 100-frame segments, with a separate model trained for each segment. Since the official implementation of Anchored-4DGS is not publicly available, we implement our own version based on the method description and experimental settings reported in the original paper.

We evaluate novel-view rendering quality using peak signal-to-noise ratio (PSNR), structural similarity index (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS). In addition to reconstruction quality, we report training time and model storage to evaluate representation efficiency. Storage is measured as the total size of the saved model, including

TABLE I
QUANTITATIVE COMPARISONS ON THE N3DV DATASET.

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Time (min) $\downarrow$	Storage (MB) $\downarrow$
4DGaussian [27]	31.05	0.921	0.081	80	48.74
ST-GS ^{/50} [35]	31.84	0.947	0.043	120	187.08
ST-GS ^{/100} [35]	31.62	0.942	0.045	70	132.39
4D-Rotor-GS [34]	31.21	0.939	0.056	100	459.34
ADC-GS [53]	29.51	0.918	0.094	35	7.42
Anchored-4DGS [44]	31.36	0.937	0.064	110	140.10
4D-Scaffold-GS [45]	31.63	0.944	0.046	90	151.12
Ours	32.07	0.949	0.045	50	44.53

TABLE II
QUANTITATIVE COMPARISONS ON THE GOOGLE IMMERSIVE DATASET.

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Time (min) $\downarrow$	Storage (MB) $\downarrow$
4DGaussian [27]	25.33	0.905	0.102	75	105.42
ST-GS ^{/50} [35]	28.03	0.906	0.091	80	243.60
ST-GS ^{/100} [35]	27.65	0.898	0.103	65	205.20
4D-Rotor-GS [34]	26.65	0.905	0.178	95	255.13
ADC-GS [53]	23.25	0.808	0.376	35	12.44
Anchored-4DGS [44]	27.22	0.882	0.123	100	126.60
4D-Scaffold-GS [45]	27.93	0.904	0.098	85	178.46
Ours	28.21	0.917	0.096	60	68.02

Gaussian or anchor parameters, temporal parameters, and the associated neural decoder weights.

D. Results

Quantitative comparison. As shown in Tables I, II, and III, ST-AGS achieves a strong balance between dynamic reconstruction quality and model compactness across all three datasets. Compared with representative primitive-level 4DGS methods, ST-AGS consistently achieves competitive or better rendering quality with substantially lower model storage. The advantage is more evident when compared with recent anchor-based 4DGS methods, where ST-AGS achieves higher PSNR

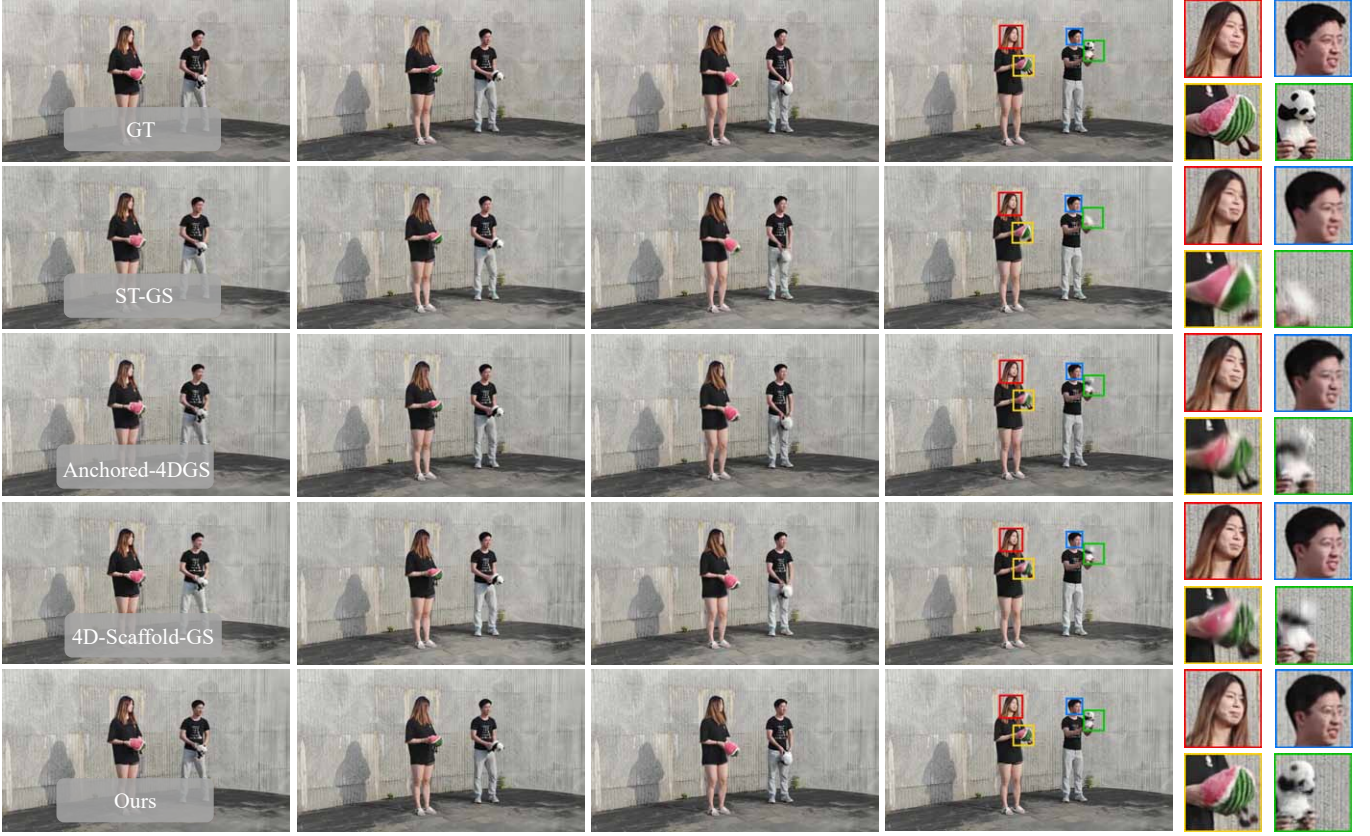


Fig. 5. Visual comparison results evaluated on ENeRF-Outdoor dataset from different comparing approaches including ST-GS [35], Anchored-4DGS [44], 4D-Scaffold-GS [45] and ours respectively.



Fig. 6. Qualitative ablation study of the proposed components, with detailed comparisons in dynamic regions. (a) Full model. (b) w/o B-Spline. (c) w/o Motion-Aware Growing. (d) w/o Staticization. (e) w/o Geometrically-Aligned Optimization.

on all three datasets while maintaining a much more compact representation. In particular, compared with 4D-Scaffold-GS [45], ST-AGS improves PSNR by an average of 0.63 dB across the three datasets while reducing the average model storage from 138.85 MB to 50.39 MB. Although ADC-GS [53] achieves lower storage through aggressive compression, its reconstruction quality is noticeably lower, especially on more challenging dynamic scenes. These results demonstrate that ST-AGS improves model compactness without sacrificing dynamic reconstruction fidelity.

Representation efficiency. The representation-scale com-

parison in Table IV further demonstrates the spatiotemporal compactness of ST-AGS. Since Gaussian-based and anchor-based methods use different basic representation units, we report the numbers of Gaussian primitives and anchors separately. Among anchor-based approaches, ST-AGS requires substantially fewer anchors than Anchored-4DGS [44] and 4D-Scaffold-GS [45] on all three datasets. In particular, ST-AGS uses only about 13%–19% as many anchors as 4D-Scaffold-GS while consistently achieving higher PSNR. This result indicates that the performance gain of ST-AGS does not come from increasing the number of anchors, but from using the

available representation capacity more effectively.

The improved representation efficiency mainly comes from the proposed Space-Time Anchor representation and Motion-Aware ST-Anchor Allocation. By allowing multiple local neural Gaussians to share the motion trajectory of a persistent ST-Anchor, ST-AGS avoids repeatedly encoding correlated temporal motion. The motion-aware allocation further concentrates additional anchors in regions with stronger reconstruction demand and more complex motion, while staticization removes unnecessary trajectory parameters from near-static regions. As a result, temporal representation capacity is concentrated on regions where dynamic modeling is actually required.

Comparison with temporal modeling and compression strategies. Different from ST-AGS, ST-GS [35] represents a long dynamic sequence using multiple temporally segmented models. Shorter temporal segments can improve local reconstruction quality, but also increase the total number of Gaussian primitives and accumulated model storage required to represent the complete sequence. In contrast, ST-AGS models the whole temporal range using a unified set of persistent ST-Anchors and continuous B-spline trajectories, leading to a more compact temporal representation.

ST-AGS also differs from existing anchor-based approaches in how representation capacity is allocated. 4D-Scaffold-GS mainly relies on reconstruction statistics and temporal coverage to guide anchor growing, whereas ST-AGS directly exploits the motion information encoded by explicit ST-Anchor trajectories together with reconstruction demand. The substantially smaller anchor population of ST-AGS suggests that these explicit motion cues provide an effective signal for identifying regions that truly require additional spatiotemporal capacity. Meanwhile, ADC-GS further compresses the model through rate-distortion optimization, adaptive quantization, and entropy coding. In contrast, the storage reduction of ST-AGS mainly originates from the compactness of the learned spatiotemporal representation itself rather than from post-hoc entropy-based compression.

Qualitative comparison. As shown in Figs. 3–5, ST-AGS preserves high-quality visual details while using a substantially more compact representation. On N3DV and Google Immersive, ST-AGS achieves visual quality comparable to or better than competing methods while requiring significantly fewer anchors. The advantage becomes more apparent on ENeRF-Outdoor, which contains larger and more challenging motion, where ST-AGS better preserves structures and local details in dynamic regions while several competing methods exhibit more noticeable blur or detail degradation. These qualitative results further show that the compactness of ST-AGS is achieved through effective spatiotemporal motion modeling and adaptive representation allocation rather than by simply reducing reconstruction capacity. Additional qualitative comparisons are provided in the supplementary material.

E. Ablation Studies

We conduct ablation studies to evaluate the key components of ST-AGS while keeping the remaining settings unchanged. Quantitative results are reported in Table V and qualitative comparisons are shown in Fig. 6.

TABLE III
QUANTITATIVE COMPARISONS ON THE ENeRF-OUTDOOR DATASET.

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Time (min) $\downarrow$	Storage (MB) $\downarrow$
4DGaussian [27]	24.25	0.717	0.218	20	59.58
ST-GS/ ⁵⁰ [35]	25.69	0.779	0.187	35	99.53
ST-GS/ ¹⁰⁰ [35]	25.17	0.776	0.185	25	49.05
4D-Rotor-GS [34]	24.63	0.652	0.489	40	361.12
ADC-GS [53]	21.59	0.489	0.505	12	13.25
Anchored-4DGS [44]	26.88	0.760	0.221	35	88.55
4D-Scaffold-GS [45]	26.08	0.801	0.107	30	86.98
Ours	27.26	0.786	0.145	15	38.62

TABLE IV
AVERAGE REPRESENTATION SCALE ON THREE DATASETS.

Method	N3DV	Google	ENeRF	Type
4DGaussian [27]	165.7K	366.1K	190.4K	Gaussian
ST-GS/ ⁵⁰ [35]	984.0K	1993.9K	654.7K	Gaussian
ST-GS/ ¹⁰⁰ [35]	913.2K	1680.6K	322.7K	Gaussian
4D-Rotor-GS [34]	576.2K	314.4K	450.3K	Gaussian
ADC-GS [53]	10.0K	14.1K	18.4K	Anchor
Anchored-4DGS [44]	299.3K	283.9K	188.4K	Anchor
4D-Scaffold-GS [45]	445.1K	525.5K	244.1K	Anchor
Ours	57.5K	80.9K	46.9K	Anchor

w/o B-Spline. We replace the cubic B-spline trajectory with a 12th-order polynomial trajectory. The polynomial is chosen to provide comparable trajectory fitting capacity to the 48-control-point B-spline, allowing us to isolate the effect of trajectory parameterization. For motion-aware allocation, the motion rate is obtained by differentiating the polynomial trajectory, while the trajectory magnitude is estimated by uniformly sampling the trajectory and accumulating consecutive displacements. Replacing the B-spline reduces PSNR from 32.07 to 30.19 and produces blurrier details in dynamic regions, indicating that the locally supported B-spline is more effective for continuous anchor motion modeling.

w/o Motion-Aware Growing. We remove the motion-aware capacity-demand prior from anchor growing, such that densification relies only on reconstruction gradients. Although this variant achieves the smallest storage of 9.33 MB, it also suffers from the most significant quality degradation, with PSNR/SSIM/LPIPS dropping to 30.15/0.935/0.060. As shown in Fig. 6, removing motion-aware growing leads to severe reconstruction blur in rapidly moving regions, where local dynamic structures cannot be properly recovered. This demonstrates that reconstruction gradients alone are insufficient to provide adequate representation capacity for complex motion, while the explicit trajectory cues effectively guide anchor allocation toward such regions.

w/o Staticization. We disable motion-aware ST-Anchor staticization, so near-static anchors retain their complete B-spline trajectories throughout training. This variant maintains similar rendering quality to the full model, but increases storage from 44.53 MB to 57.66 MB. The relatively small visual difference further confirms that staticization mainly

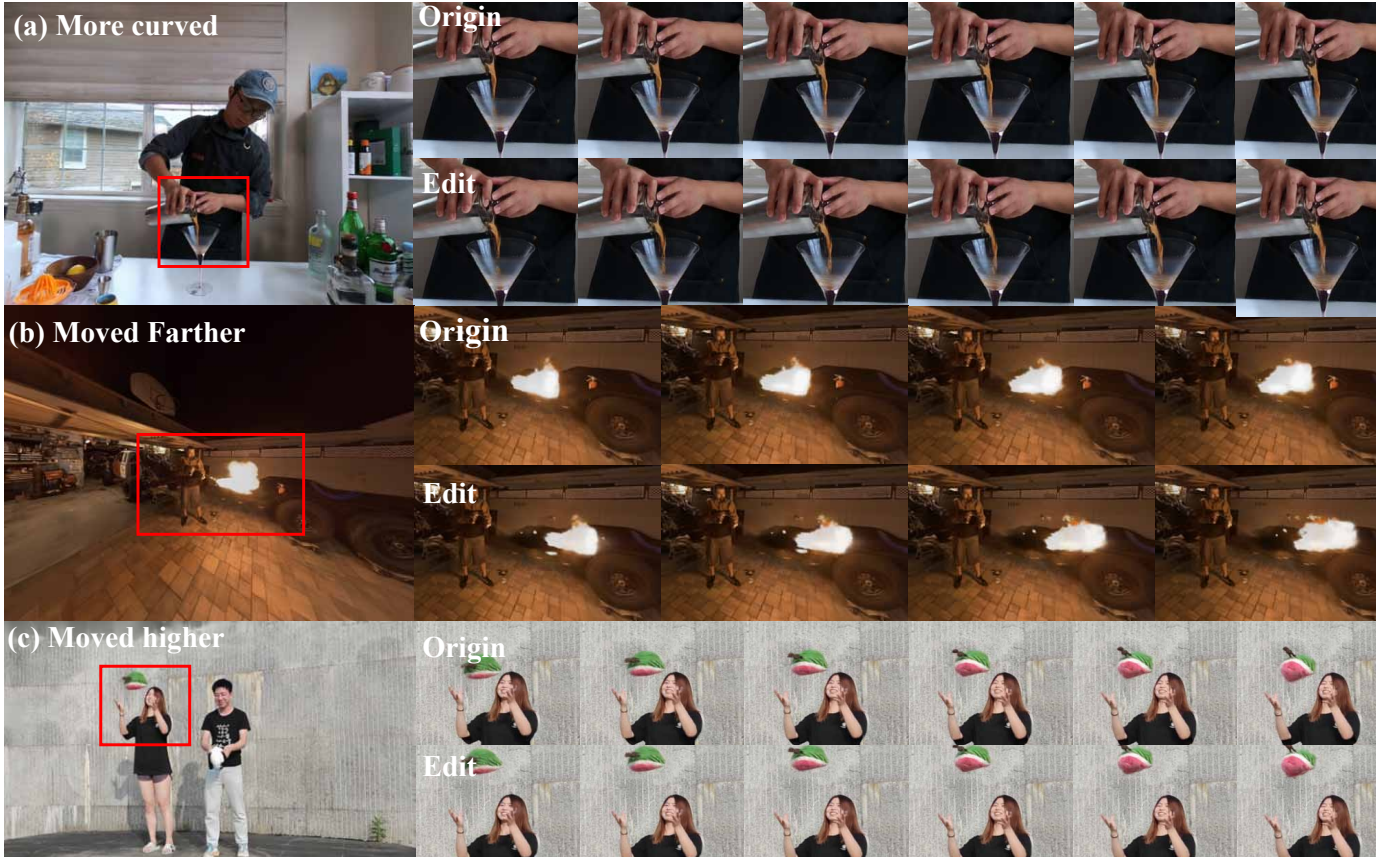


Fig. 7. **Controllable motion editing with ST-AGS.** (a) increasing trajectory curvature, (b) extending the motion range, and (c) raising the motion trajectory.

TABLE V
QUANTITATIVE ABLATION STUDY OF THE PROPOSED COMPONENTS ON
THE N3DV DATASET.

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Storage (MB) $\downarrow$
w/o B-Spline	30.19	0.937	0.054	32.48
w/o Motion-Aware Growing	30.15	0.935	0.060	9.33
w/o Staticization	31.82	0.947	0.046	57.66
w/o Geo.-Aligned Opt.	31.68	0.945	0.047	45.46
Full	32.07	0.949	0.045	44.53

improves spatiotemporal compactness by removing redundant temporal parameters from near-static anchors.

w/o Geometrically-Aligned Optimization. We remove both the geometric alignment loss $\mathcal{L}_{\text{align}}$ and the geometry-guided photometric loss $\mathcal{L}_{\text{dRGB}}$, leaving only the standard photometric reconstruction objective. This reduces PSNR from 32.07 to 31.68 and leads to less distinct local structures in dynamic regions. These results indicate that the proposed geometric guidance improves the spatial organization of local neural Gaussians and further enhances reconstruction fidelity under the compact representation.

Overall, the full ST-AGS achieves the best balance between reconstruction quality and representation compactness.

F. Application

Controllable Motion Editing. The explicit B-spline trajectories of ST-Anchors provide an intuitive interface for controllable motion editing without introducing an additional editing module. Since each dynamic ST-Anchor is explicitly parameterized by B-spline control points, users can directly manipulate its motion trajectory.

Given a screen-space dragging constraint at a reference timestamp t^* , we first lift the 2D displacement to a 3D displacement $\Delta \mathbf{a}_i$ on the corresponding camera viewing plane. Since a cubic B-spline at t^* depends only on four neighboring control points, we distribute the desired displacement by solving

$$\min_{\{\Delta \mathbf{c}_{i,s+\ell}\}} \sum_{\ell=0}^3 \|\Delta \mathbf{c}_{i,s+\ell}\|_2^2 \quad \text{s.t.} \quad \sum_{\ell=0}^3 B_{\ell}(u^*) \Delta \mathbf{c}_{i,s+\ell} = \Delta \mathbf{a}_i. \quad (9)$$

This minimum-norm update restricts the modification to locally supported control points, so the editing influence remains within a local temporal neighborhood and smoothly transitions to the unedited trajectory, preserving temporal continuity.

As shown in Fig. 7, we demonstrate three representative manipulations: increasing trajectory curvature, extending the motion range, and raising the motion trajectory, illustrating the flexible editing capability of the explicit ST-Anchor representation.

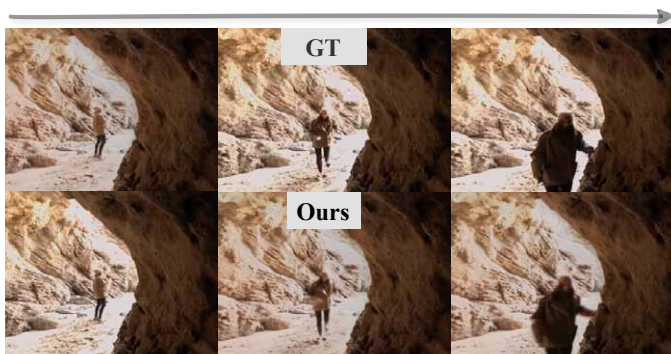


Fig. 8. A representative failure case of ST-AGS. When the person moves from a distant region toward the camera, large motion together with significant scale changes may result in local blur in this challenging case.

V. CONCLUSION AND LIMITATION

We present ST-AGS, a compact 4D Gaussian splatting framework based on Space-Time Anchors, which explicitly model continuous anchor motion to reduce temporal redundancy in dynamic scene representations. Experiments on N3DV, Google Immersive, and ENeRF-Outdoor show that ST-AGS achieves competitive or superior novel-view synthesis quality with substantially fewer anchors and lower model storage than recent state-of-the-art anchor-based 4DGS approaches, while the explicit ST-Anchor representation also shows potential for controllable trajectory-based motion editing. We hope that ST-AGS can inspire subsequent works on more compact, effective, and controllable 4D Gaussian splatting frameworks for dynamic scene reconstruction.

Despite these advantages, ST-AGS may still produce local blur in challenging cases involving large motion and significant scale changes, as shown in Fig. 8. The current B-spline trajectory representation is better suited to smooth and continuous motion, which may limit its flexibility in handling such challenging motion patterns. Future work will explore more flexible motion representations to further improve robustness in such dynamic scenes.

REFERENCES

- [1] E. De Aguiar, C. Stoll, C. Theobalt, N. Ahmed, H.-P. Seidel, and S. Thrun, "Performance capture from sparse multi-view video," in *ACM SIGGRAPH*, 2008, pp. 1–10.
- [2] A. Collet, M. Chuang, P. Sweeney, D. Gillett, D. Evseev, D. Calabrese, H. Hoppe, A. Kirk, and S. Sullivan, "High-quality streamable free-viewpoint video," *ACM TOG*, vol. 34, no. 4, pp. 1–13, 2015.
- [3] H.-X. Song, J. Huang, Y.-P. Cao, and T.-J. Mu, "Hdr-net-fusion: Real-time 3d dynamic scene reconstruction with a hierarchical deep reinforcement network," *Computational Visual Media*, vol. 7, no. 4, pp. 419–435, 2021.
- [4] Z. Xu, S. Bi, K. Sunkavalli, S. Hadap, H. Su, and R. Ramamoorthi, "Deep view synthesis from sparse photometric images," *ACM TOG*, vol. 38, no. 4, pp. 1–13, 2019.
- [5] C. Gao, A. Saraf, J. Kopf, and J.-B. Huang, "Dynamic view synthesis from dynamic monocular video," in *IEEE ICCV*, 2021, pp. 5712–5721.
- [6] J. Zhang, X. Liu, X. Ye, F. Zhao, Y. Zhang, M. Wu, Y. Zhang, L. Xu, and J. Yu, "Editable free-viewpoint video using a layered neural representation," *ACM TOG*, vol. 40, no. 4, pp. 1–18, 2021.
- [7] A. Trevithick, M. Chan, M. Stengel, E. Chan, C. Liu, Z. Yu, S. Khamis, M. Chandraker, R. Ramamoorthi, and K. Nagano, "Real-time radiance fields for single-image portrait view synthesis," *ACM TOG*, vol. 42, no. 4, pp. 1–15, 2023.
- [8] D. Li, S.-S. Huang, T. Shen, and H. Huang, "Dynamic view synthesis with spatio-temporal feature warping from sparse views," in *ACM MM*, 2023, pp. 1565–1576.
- [9] S. Qian, T. Kirschstein, L. Schoneveld, D. Davoli, S. Giebenhain, and M. Nießner, "Gaussianavatars: Photorealistic head avatars with rigged 3d gaussians," in *IEEE CVPR*, 2024.
- [10] L. Hu, H. Zhang, Y. Zhang, B. Zhou, B. Liu, S. Zhang, and L. Nie, "Gaussianavatar: Towards realistic human avatar modeling from a single video via animatable 3d gaussians," in *IEEE CVPR*, 2024.
- [11] J. Xiang, X. Gao, Y. Guo, and J. Zhang, "Flashavatar: High-fidelity head avatar with efficient gaussian embedding," in *IEEE CVPR*, 2024.
- [12] Y. Xu, B. Chen, Z. Li, H. Zhang, L. Wang, Z. Zheng, and Y. Liu, "Gaussian head avatar: Ultra high-fidelity head avatar via dynamic gaussians," in *IEEE CVPR*, 2024.
- [13] Z. Shao, Z. Wang, Z. Li, D. Wang, X. Lin, Y. Zhang, M. Fan, and Z. Wang, "SplattingAvatar: Realistic Real-Time Human Avatars with Mesh-Embedded Gaussian Splatting," in *IEEE CVPR*, June 2024, pp. 1606–1616.
- [14] Z. Fan, S.-S. Huang, Y. Zhang, D. Shang, J. Zhang, Y. Guo, and H. Huang, "Rgavatar: Relightable 4d gaussian avatar from monocular videos," *IEEE TVCG*, 2025.
- [15] Y. Zhang, J. Yang, Z. Liu, R. Wang, G. Chen, X. Tong, and B. Guo, "Virtualcube: An immersive 3d video communication system," *IEEE TVCG*, vol. 28, no. 5, pp. 2146–2156, 2022.
- [16] H. Tu, R. Shao, X. Dong, S. Zheng, H. Zhang, L. Chen, M. Wang, W. Li, S. Ma, S. Zhang *et al.*, "Tele-aloha: A telepresence system with low-budget and high-authenticity using sparse rgb cameras," in *ACM SIGGRAPH Conference Papers*, 2024, pp. 1–12.
- [17] Y. Shen, W. Liang, and J. Ma, "Liteat: A data-lightweight and user-adaptive vr telepresence system for remote education," *IEEE TVCG*, 2025.
- [18] C. L. Zitnick, S. B. Kang, M. Uyttendaele, S. Winder, and R. Szeliski, "High-quality video view interpolation using a layered representation," *ACM TOG*, vol. 23, no. 3, pp. 600–608, 2004.
- [19] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [20] A. Pumarola, E. Corona, G. Pons-Moll, and F. Moreno-Noguer, "D-nerf: Neural radiance fields for dynamic scenes," in *IEEE CVPR*, 2021, pp. 10 318–10 327.
- [21] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM TOG*, vol. 41, no. 4, pp. 1–15, 2022.
- [22] A. Chen, Z. Xu, A. Geiger, J. Yu, and H. Su, "Tensorf: Tensorial radiance fields," in *ECCV*, 2022, pp. 333–350.
- [23] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa, "Plenoxels: Radiance fields without neural networks," in *IEEE CVPR*, 2022, pp. 5501–5510.
- [24] Z. Chen, T. Funkhouser, P. Hedman, and A. Tagliasacchi, "Mobilenetf: Exploiting the polygon rasterization pipeline for efficient neural field rendering on mobile architectures," in *IEEE CVPR*, 2023, pp. 16 569–16 578.
- [25] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis *et al.*, "3d gaussian splatting for real-time radiance field rendering," *ACM TOG*, vol. 42, no. 4, pp. 139–1, 2023.
- [26] T. Wu, Y.-J. Yuan, L.-X. Zhang, J. Yang, Y.-P. Cao, L.-Q. Yan, and L. Gao, "Recent advances in 3d gaussian splatting," *Computational Visual Media*, vol. 10, no. 4, pp. 613–642, 2024.
- [27] G. Wu, T. Yi, J. Fang, L. Xie, X. Zhang, W. Wei, W. Liu, Q. Tian, and X. Wang, "4d gaussian splatting for real-time dynamic scene rendering," in *IEEE CVPR*, 2024, pp. 20 310–20 320.
- [28] D. Li, S.-S. Huang, Z. Lu, X. Duan, and H. Huang, "St-4dgs: Spatial-temporally consistent 4d gaussian splatting for efficient dynamic scene rendering," in *ACM SIGGRAPH Conference Papers*, 2024, pp. 1–11.
- [29] X. Zhang, Z. Liu, Y. Zhang, X. Ge, D. He, T. Xu, Y. Wang, Z. Lin, S. Yan, and J. Zhang, "Mega: Memory-efficient 4d gaussian splatting for dynamic scenes," in *IEEE ICCV*, 2025, pp. 27 828–27 838.
- [30] D. Li, S.-S. Huang, and H. Huang, "Mpgs: Multi-plane gaussian splatting for compact scenes rendering," *IEEE TVCG*, 2025.
- [31] Y. Chang, H. Xu, J. Li, X. Wang, Y. Tian, and P. Peng, "Kpgs: Towards real-world complex dynamic scene rendering with keyframe-driven predictable gaussian splatting," *IEEE TVCG*, 2026.
- [32] J. Lee, C. Won, H. Jung, I. Bae, and H.-G. Jeon, "Fully explicit dynamic gaussian splatting," in *NeurIPS*, 2024.

- [33] T. Kang, J. Park, K. Lee, and Y. Lee, "Clustered error correction with grouped 4d gaussian splatting," in *SIGGRAPH Asia Conference Papers*, December 2025, pp. 1–12.
- [34] Y. Duan, F. Wei, Q. Dai, Y. He, W. Chen, and B. Chen, "4d-rotor gaussian splatting: Towards efficient novel view synthesis for dynamic scenes," in *ACM SIGGRAPH*, July 2024.
- [35] Z. Li, Z. Chen, Z. Li, and Y. Xu, "Spacetime gaussian feature splatting for real-time dynamic view synthesis," in *IEEE CVPR*, 2024, pp. 8508–8520.
- [36] Y. Lin, Z. Dai, S. Zhu, and Y. Yao, "Gaussian-flow: 4d reconstruction with dynamic 3d gaussian particle," in *IEEE CVPR*, 2024, pp. 20542–20551.
- [37] Y. Wang, P. Yang, Z. Xu, J. Sun, Z. Zhang, Y. Chen, H. Bao, S. Peng, and X. Zhou, "Freetimegs: Free gaussian primitives at anytime anywhere for dynamic scene reconstruction," in *IEEE CVPR*, 2025.
- [38] Z. Liao and Y. Liu, "Sharptimegs: Sharp and stable dynamic gaussian splatting via lifespan modulation," in *IEEE CVPR*, 2026.
- [39] M. Lee, B. Lee, L. Y. Lee, E. Lee, S. Kim, S. Song, J. C. Lee, J. H. Ko, J. Park, and E. Park, "Optimized minimal 4d gaussian splatting," *arXiv preprint arXiv:2510.03857*, 2025.
- [40] P. Dai, P. Zhang, Z. Dong, K. Xu, Y. Peng, D. Ding, Y. Shen, Y. Yang, X. Liu, and W. Xu, "4d gaussian videos with motion layering," *ACM TOG*, vol. 44, no. 4, 2025.
- [41] T. Lu, M. Yu, L. Xu, Y. Xiangli, L. Wang, D. Lin, and B. Dai, "Scaffold-gs: Structured 3d gaussians for view-adaptive rendering," in *IEEE CVPR*, 2024, pp. 20654–20664.
- [42] D. Li, S.-S. Huang, H. Fu, and H. Huang, "Gang: Geometrically-aligned neural gaussians for efficient and realistic relighting," *IEEE TVCG*, 2026.
- [43] S. Kwak, J. Kim, J. Y. Jeong, W.-S. Cheong, J. Oh, and M. Kim, "Modex-gs: Global-to-local motion decomposition and temporal interval adjustment for compact dynamic 3d gaussian splatting," in *IEEE CVPR*, 2025, pp. 20892–20901.
- [44] Y. Li, B. Pang, Y. Chen, and G. Wang, "Anchored 4d gaussian splatting for dynamic novel view synthesis," in *ACM SIGGRAPH Asia Conference Papers*, 2025, pp. 1–11.
- [45] W. O. Cho, I. Cho, S. Kim, J. Bae, Y. Uh, and S. J. Kim, "4d scaffold gaussian splatting with dynamic-aware anchor growing for efficient and high-fidelity dynamic scene reconstruction," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 5, 2026, pp. 3363–3371.
- [46] R. Zhu, Y. Liang, H. Chang, J. Deng, J. Lu, W. Yang, T. Zhang, and Y. Zhang, "Motiongs: Exploring explicit motion guidance for deformable 3d gaussian splatting," *NeurIPS*, vol. 37, pp. 101790–101817, 2024.
- [47] Z. Gao, B. Planche, M. Zheng, A. Choudhuri, T. Chen, and Z. Wu, "7dgs: Unified spatial-temporal-angular gaussian splatting," in *IEEE ICCV*, October 2025, pp. 14562–14571.
- [48] Y.-H. Huang, Y.-T. Sun, Z. Yang, X. Lyu, Y.-P. Cao, and X. Qi, "Sc-gs: Sparse-controlled gaussian splatting for editable dynamic scenes," in *IEEE CVPR*, 2024, pp. 4220–4230.
- [49] J. Wacziarg, P. Borycki, J. Kaleta, S. Tadeja, and P. Spurek, "D-miso: Editing dynamic 3d scenes using multi-gaussians soup," *NeurIPS*, vol. 37, pp. 107865–107889, 2024.
- [50] T. Li, M. Slavcheva, M. Zollhoefer, S. Green, C. Lassner, C. Kim, T. Schmidt, S. Lovegrove, M. Goesele, R. Newcombe *et al.*, "Neural 3d video synthesis from multi-view video," in *IEEE CVPR*. IEEE, 2022, pp. 5511–5521.
- [51] M. Broxton, J. Flynn, R. Overbeck, D. Erickson, P. Hedman, M. Duvall, J. Dourgarian, J. Busch, M. Whalen, and P. Debevec, "Immersive light field video with a layered mesh representation," *ACM TOG*, vol. 39, no. 4, pp. 86–1, 2020.
- [52] H. Lin, S. Peng, Z. Xu, Y. Yan, Q. Shuai, H. Bao, and X. Zhou, "Efficient neural radiance fields for interactive free-viewpoint video," in *ACM SIGGRAPH Asia conference papers*, 2022, pp. 1–9.
- [53] H. Huang, Q. Yang, M. Liu, Y. Xu, and Z. Li, "Adc-gs: Anchor-driven deformable and compressed gaussian splatting for dynamic scene reconstruction," *arXiv preprint arXiv:2505.08196*, 2025.
- [54] S.-S. Huang, S.-K. Zhang, S. Yang, J.-W. Guo, and H. Huang, "A survey of recent advances in generative 3d reconstruction," *Journal of Computer Science and Technology*, vol. 40, no. 5, pp. 1236–1254, 2025.
- [55] J. Zhu and H. Tang, "Dynamic scene reconstruction: Recent advance in real-time rendering and streaming," *arXiv preprint arXiv:2503.08166*, 2025.
- [56] J. Artois, M. Courteaux, G. Wallendaël, and P. Lambert, "Opendibr: Open real-time depth-image-based renderer of light field videos for vr," *Multimedia Tools and Applications*, vol. 83, pp. 25797–25815, 08 2023.
- [57] M. Guo, J. Hou, J. Jin, H. Liu, H. Zeng, and J. Lu, "Content-aware warping for view synthesis," *IEEE TPAMI*, 2023.
- [58] A. Cao, C. Rockwell, and J. Johnson, "Fwd: Real-time novel view synthesis with forward warping and depth," in *IEEE CVPR*, 2022, pp. 15713–15724.
- [59] W. Xian, J.-B. Huang, J. Kopf, and C. Kim, "Space-time neural irradiance fields for free-viewpoint video," in *IEEE CVPR*, 2021, pp. 9421–9431.
- [60] M. İşik, M. Rünz, M. Georgopoulos, T. Khakhulin, J. Starck, L. Agapito, and M. Nießner, "Humanrf: High-fidelity neural radiance fields for humans in motion," *ACM TOG*, vol. 42, no. 4, pp. 1–12, 7 2023.
- [61] H. Lin, Z. Xu, S. Peng, L. Chen, Y. Xu, X. Zhou, and H. Bao, "Im4d: High-fidelity and real-time novel view synthesis for dynamic scenes," *ACM TOG*, vol. 42, no. 6, pp. 1–14, 12 2023.
- [62] K. Park, U. Sinha, J. T. Barron, S. Bouaziz, D. B. Goldman, S. M. Seitz, and R. Martin-Brualla, "Nerfies: Deformable neural radiance fields," in *IEEE ICCV*, 2021, pp. 5865–5874.
- [63] K. Park, U. Sinha, P. Hedman, J. T. Barron, S. Bouaziz, D. B. Goldman, R. Martin-Brualla, and S. M. Seitz, "Hypernerf: A higher-dimensional representation for topologically varying neural radiance fields," *ACM TOG*, vol. 40, no. 6, p. 238, dec 2021.
- [64] L. Li, Z. Shen, Z. Wang, L. Shen, and P. Tan, "Streaming radiance fields for 3d video synthesis," in *NeurIPS*, 2022.
- [65] L. Song, A. Chen, Z. Li, Z. Chen, L. Chen, J. Yuan, Y. Xu, and A. Geiger, "Nerfplayer: A streamable dynamic scene representation with decomposed neural radiance fields," *IEEE TVCG*, vol. 29, no. 5, pp. 2685–2694, 2023.
- [66] J. Li, Z. Feng, Q. She, H. Ding, C. Wang, and G. H. Lee, "Mine: Towards continuous depth mpi with nerf for novel view synthesis," in *IEEE ICCV*, 2021, pp. 14317–14326.
- [67] A. Cao, K. Wei, J. Yang, Y. Chen, X. Tong, and Y. Wang, "Hexplane: A fast representation for dynamic scenes," in *IEEE CVPR*, 2023, pp. 130–141.
- [68] P. Wang, L. Liu, Y. Liu, C. Theobalt, T. Komura, and W. Wang, "Neus: Learning neural implicit surfaces by volume rendering for multi-view reconstruction," in *NeurIPS*, vol. 34, 2021, pp. 27171–27183.
- [69] S. Izadi, R. A. Newcombe, D. Kim, O. Hilliges, D. Molyneaux, S. Hodges, P. Kohli, J. Shotton, A. J. Davison, and A. Fitzgibbon, "Kinectfusion: Real-time 3d reconstruction and interaction using a moving depth camera," in *ACM UIST*, 2011, pp. 559–568.
- [70] R. A. Newcombe, S. Izadi, O. Hilliges, D. Kim, A. J. Davison, P. Kohli, J. Shotton, S. Hodges, and A. Fitzgibbon, "Dynamicfusion: Reconstruction and tracking of non-rigid scenes in real-time," in *IEEE CVPR*, 2015, pp. 2012–2021.
- [71] M. Dou, S. Khamis, Y. Degtyarev, P. Davidson, S. R. Fanello, A. Kowdle, S. O. Escolano, C. Rhemann, D. Kim, J. Taylor, P. Kohli, V. Tankovich, and S. Izadi, "Fusion4d: Real-time performance capture of challenging scenes," *ACM TOG*, vol. 35, no. 4, pp. 1–13, 2016.
- [72] T. Yu, Z. Zheng, K. Guo, J. Liu, Q. Cui, and Y. Liu, "Doublefusion: Real-time capturing of human performance with inner body and surface details," *ACM TOG*, vol. 37, no. 6, pp. 1–15, 2018.
- [73] T. Yu, K. Guo, F. Xu, Y. Dong, Z. Su, J. Liu, J. Li, Q. Dai, and Y. Liu, "Bodyfusion: Real-time capture of human motion and surface geometry using a single depth camera," in *IEEE ICCV*, 2017, pp. 5888–5897.
- [74] W. Mao, R. Hartley, M. Salzmann, and M. Liu, "Neural sdf flow for 3d reconstruction of dynamic scenes," in *ICLR*, 2024.
- [75] S. Wu, T. Jakab, C. Rupprecht, and A. Vedaldi, "Dove: Learning deformable 3d objects by watching videos," *International Journal of Computer Vision*, vol. 131, no. 10, pp. 2623–2634, 2023.
- [76] I. Liu, H. Su, and X. Wang, "Dynamic gaussians mesh: Consistent mesh reconstruction from dynamic scenes," in *ICLR*, 2025.
- [77] S. Ma, Y. Luo, W. Yang, and Y. Yang, "Mags: Reconstructing and simulating dynamic 3d objects with mesh-adsorbed gaussian splatting," in *IEEE ICCV*, 2025, pp. 14583–14593.
- [78] X. Li, J. Tong, J. Hong, V. Rolland, and L. Petersson, "Dgns: Deformable gaussian splatting and dynamic neural surface for monocular dynamic 3d reconstruction," in *ACM MM*, 2025, pp. 1812–1821.
- [79] C. Zheng, L. Xue, J. Zarate, and J. Song, "GauStar: Gaussian surface tracking and reconstruction," in *IEEE CVPR*, 2025, pp. 16543–16553.
- [80] G. Pavlakos, V. Choutas, N. Ghorbani, T. Bolkart, A. A. Osman, D. Tzionas, and M. J. Black, "Expressive body capture: 3d hands, face, and body from a single image," in *IEEE CVPR*, 2019, pp. 10967–10977.
- [81] J. He, C. H. Lin, L. Qi, and M.-H. Yang, "Restage4d: Reanimating deformable 3d reconstruction from a single video," in *NeurIPS*, 2025.
- [82] S. Shin, J. Park, and S. Cho, "Locality-aware gaussian compression for fast and high-quality rendering," in *ICLR*, 2025, pp. 28904–28931.
- [83] J. C. Lee, D. Rho, X. Sun, J. H. Ko, and E. Park, "Compact 3d gaussian representation for radiance field," in *IEEE CVPR*, 2024, pp. 21719–21728.

ST-AGS: Space-Time Anchored 4D Gaussian Splatting for Geometrically-Aligned Dynamic Scene Reconstruction

Supplementary Material

I. ADDITIONAL METHOD DETAILS

This section provides additional formulation and implementation details omitted from the main paper for clarity. We follow the same organization as the main method and focus on the exact computation of the Space-Time Anchor representation, Motion-Aware ST-Anchor Allocation, and Geometrically-Aligned 4D Optimization.

A. Additional Details of Space-Time Anchor Representation

Dynamic ST-Anchor parameterization. A dynamic ST-Anchor consists of B-spline control points $\mathbf{C}_i = \{\mathbf{c}_{i,j}\}_{j=0}^{M-1}$, an anchor feature $\mathbf{f}_i$, local offsets $\mathbf{O}_i = \{\mathbf{o}_{ik}\}_{k=1}^K$, and spatial attributes including scaling, rotation, and opacity. We use $M = 48$ control points and generate at most $K = 10$ local neural Gaussians from each ST-Anchor. For notational simplicity, the main paper absorbs the anchor-level spatial scaling into the local offset $\mathbf{o}_{ik}$. In the actual implementation, the normalized local offset is explicitly modulated by a learnable anchor-level spatial scale $\mathbf{s}_i^{\text{off}}$, yielding

$$\mathbf{x}_{ik}(t) = \mathbf{a}_i(t) + \mathbf{s}_i^{\text{off}} \odot \mathbf{o}_{ik}. \quad (1)$$

Temporal parameterization. The frame timestamp is normalized to $t \in [0, 1]$. For a cubic B-spline with M control points, the trajectory contains $M - 3$ temporal segments. We map t to the active segment and its local coordinate by

$$q = (M - 3)t, \quad s = \min(\lfloor q \rfloor, M - 4), \quad u = q - s. \quad (2)$$

With $M = 48$, the trajectory contains 45 cubic segments. The cubic B-spline basis functions used in our implementation are

$$\begin{aligned} B_0(u) &= \frac{(1-u)^3}{6}, \\ B_1(u) &= \frac{3u^3 - 6u^2 + 4}{6}, \\ B_2(u) &= \frac{-3u^3 + 3u^2 + 3u + 1}{6}, \\ B_3(u) &= \frac{u^3}{6}. \end{aligned} \quad (3)$$

Trajectory initialization. All B-spline control points are initialized with the initial ST-Anchor position,

$$\mathbf{c}_{i,j} = \mathbf{a}_i^0, \quad j = 0, \dots, M - 1. \quad (4)$$

Since the cubic B-spline basis forms a partition of unity, this initialization yields a stationary trajectory over the complete

temporal range. The control points are subsequently optimized through multi-view reconstruction supervision.

B. Additional Details of Motion-Aware ST-Anchor Allocation

1) Reconstruction Demand and Motion Prior: For each neural Gaussian candidate (i, k) , the reconstruction demand is computed from the average magnitude of the training-loss gradient with respect to its projected 2D center:

$$R_{ik} = \frac{1}{N_{ik}} \sum_{n=1}^{N_{ik}} \left\| \nabla_{\mu_{ik}^{2D}} \mathcal{L}^{(n)} \right\|_2, \quad (5)$$

where N_{ik} denotes the number of valid observations used for statistics accumulation.

The trajectory magnitude of ST-Anchor i is measured by the accumulated displacement between neighboring control points,

$$m_i = \sum_{j=0}^{M-2} \left\| \mathbf{c}_{i,j+1} - \mathbf{c}_{i,j} \right\|_2, \quad (6)$$

while the motion rate is computed from the derivative magnitude of the continuous B-spline trajectory,

$$v_i = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \left\| \frac{\partial \mathbf{a}_i(t)}{\partial t} \right\|_2. \quad (7)$$

We analytically evaluate the B-spline derivative and average it over five uniformly sampled timestamps. To reduce the influence of motion outliers, we use the 90th percentile of m_i over the current dynamic ST-Anchor population as the scene-level reference m_{ref} and compute

$$\hat{m}_i = \text{clip} \left(\frac{m_i}{m_{\text{ref}}}, 0, 2 \right). \quad (8)$$

The motion-aware capacity prior and final growing score are then given by

$$\rho_i = (1 + \lambda_m \hat{m}_i) e^{\gamma v_i}, \quad (9)$$

$$S_{ik} = \rho_i R_{ik}, \quad S_{ik} > \tau_{\text{grow}}. \quad (10)$$

2) *Trajectory-Preserving Anchor Growing*: For each selected growing candidate, a representative timestamp t^* is determined from its accumulated reconstruction evidence. The candidate location is obtained from the corresponding local neural Gaussian:

$$\mathbf{y}_{ik} = \mathbf{a}_i(t^*) + \mathbf{s}_i^{\text{off}} \odot \mathbf{o}_{ik}. \quad (11)$$

When candidate (i, k) is instantiated as a child ST-Anchor i' , we compute the translation

$$\Delta_{i'} = \mathbf{y}_{ik} - \mathbf{a}_i(t^*), \quad (12)$$

and initialize its control points by

$$\mathbf{c}_{i',j} = \mathbf{c}_{i,j} + \Delta_{i'}, \quad j = 0, \dots, M-1. \quad (13)$$

Applying the same translation to all control points preserves the temporal shape of the parent trajectory while placing the child at the selected spatial location.

3) *ST-Anchor Staticization*: After the trajectories are sufficiently optimized, ST-Anchors with $m_i < \tau_{\text{sta}}$ are treated as near-static. We uniformly sample $N_t = 100$ positions along the learned trajectory and compute the temporal centroid

$$\bar{\mathbf{a}}_i = \frac{1}{N_t} \sum_{n=1}^{N_t} \mathbf{a}_i(t_n). \quad (14)$$

The B-spline trajectory is then replaced by the fixed position $\bar{\mathbf{a}}_i$. Accordingly, the dynamic parameterization $(\mathbf{C}_i, \mathbf{O}_i, \mathbf{f}_i)$ is converted to the static parameterization $(\bar{\mathbf{a}}_i, \mathbf{O}_i, \mathbf{f}_i)$, while the remaining spatial attributes are retained. The corresponding B-spline control points and optimizer states are removed from the learnable parameter set. The time-conditioned Gaussian decoder remains unchanged.

C. Additional Details of Geometrically-Aligned 4D Optimization

1) *Local Gaussian Depth Consistency*: For the geometric alignment branch, we detach the ST-Anchor trajectory when constructing the local Gaussian center:

$$\tilde{\mathbf{x}}_{ik}(t) = \text{sg}(\mathbf{a}_i(t)) + \mathbf{s}_i^{\text{off}} \odot \mathbf{o}_{ik}, \quad (15)$$

where $\text{sg}(\cdot)$ denotes stop-gradient. The camera-space depth d_{ik} and projected image position $\mathbf{p}_{ik}$ are obtained by projecting $\tilde{\mathbf{x}}_{ik}(t)$ using the corresponding camera parameters.

The depth consistency reference is sampled from the detached rasterized depth map:

$$\hat{d}_{ik} = \text{Sample}(\text{sg}(D), \mathbf{p}_{ik}). \quad (16)$$

Only local Gaussians with valid in-frustum projections, positive sampled depth, and sufficiently small relative depth discrepancy are included in the valid set $\mathcal{V}$. Since both the rendered depth reference and ST-Anchor trajectory are detached, gradients from $\mathcal{L}_{\text{align}}$ are restricted to the anchor-relative local geometry, including the local offsets and spatial scales, without directly modifying the learned B-spline trajectory.

2) *Error Splatting*: For each valid local Gaussian, we define its relative geometric discrepancy as

$$e_{ik} = \frac{|d_{ik} - \hat{d}_{ik}|}{\hat{d}_{ik} + \varepsilon}. \quad (17)$$

We regard e_{ik} as a scalar geometric error associated with Gaussian (i, k) and splat it onto the image plane according to its visible rendering contribution. Specifically, the contribution of Gaussian (i, k) to pixel $\mathbf{p}$ is defined as

$$w_{ik}(\mathbf{p}) = T_{ik}(\mathbf{p})\alpha_{ik}(\mathbf{p}), \quad (18)$$

where $T_{ik}(\mathbf{p})$ denotes the accumulated transmittance before the Gaussian and $\alpha_{ik}(\mathbf{p})$ denotes its opacity contribution at pixel $\mathbf{p}$. We then construct the geometry inconsistency map through contribution-weighted Error Splatting:

$$E(\mathbf{p}) = \frac{\sum_{(i,k) \in \mathcal{V}} w_{ik}(\mathbf{p})e_{ik}}{\sum_{(i,k) \in \mathcal{V}} w_{ik}(\mathbf{p}) + \varepsilon}. \quad (19)$$

The normalization removes the dependence on the number of Gaussians contributing to each pixel, such that $E(\mathbf{p})$ reflects the contribution-weighted geometric inconsistency of the visible Gaussian representation rather than representation density. The resulting map therefore provides an image-space geometric signal that can be directly used to guide photometric optimization.

We convert the geometry inconsistency map into the photometric weighting map

$$\mathbf{W}_d = \mathbf{1} + \beta \mathbf{E}, \quad (20)$$

which is used in the geometry-guided photometric loss $\mathcal{L}_{\text{dRGB}}$ defined in the main paper. Pixels associated with larger geometric inconsistency receive larger weights, allowing the photometric objective to place stronger emphasis on regions where the current local Gaussian geometry remains less well aligned.

II. DETAILS OF EXPERIMENT

Optimization setup. We adopt a unified optimization strategy for all experiments. The major ST-Anchor parameters and neural decoders are optimized using exponentially decayed learning rates over the complete 120k training schedule, while opacity, scaling, and rotation use fixed learning rates. The detailed learning-rate configuration of the major ST-Anchor parameters and neural Gaussian decoders is summarized in Table I. All three datasets are trained for 120k iterations using the same basic optimization schedule. For the loss configuration, we set $\lambda_{\text{DSSIM}} = 0.2$. The weights of $\mathcal{L}_{\text{align}}$ and $\mathcal{L}_{\text{dRGB}}$ are selected according to scene characteristics, with $\lambda_{\text{align}} \in [10^{-4}, 5 \times 10^{-3}]$ and $\lambda_{\text{dRGB}} \in [5 \times 10^{-3}, 10^{-1}]$.

Anchor growing and ST-Anchor staticization. At the early training stage, we first accumulate reconstruction statistics and start ST-Anchor growing from 6k iterations. Growing is then performed every 6k iterations and is stopped after 60k iterations, leaving sufficient iterations for subsequent representation refinement. The base growing threshold is set to $\tau_{\text{grow}} = 1 \times 10^{-3}$. For motion-aware capacity allocation, we

TABLE I
LEARNING-RATE CONFIGURATION USED IN OUR EXPERIMENTS.

Parameter	Initial LR	Final LR
Gaussian offset	1.0×10^{-2}	1.0×10^{-4}
Anchor feature	7.5×10^{-3}	7.5×10^{-4}
B-spline control points	1.0×10^{-3}	1.0×10^{-5}
Opacity MLP	2.0×10^{-3}	2.0×10^{-5}
Covariance MLP	4.0×10^{-3}	4.0×10^{-4}
Color MLP	8.0×10^{-3}	5.0×10^{-5}

set $\lambda_m = 1.0$ and $\gamma = 0.3$ to control the influence of trajectory magnitude and motion rate on the growing score. The exact growing schedule is slightly adjusted according to the training configuration of different scenes, while the growing criterion and motion-aware weighting formulation remain unchanged.

At the middle-to-late training stage, we further perform ST-Anchor staticization to reduce redundant temporal parameters in near-static regions. For each dynamic ST-Anchor, it is regarded as near-static when its trajectory magnitude satisfies $m_i < \tau_{\text{sta}}$. The staticization threshold is defined according to the spatial representation scale as

$$\tau_{\text{sta}} = s_{\text{sta}} \min(v_{\text{voxel}}, 0.001), \quad (21)$$

where $v_{\text{voxel}} = 0.001$ and s_{sta} controls the staticization strength. According to the motion range and dynamic complexity of different scenes, s_{sta} is set between 500 and 2500, corresponding to τ_{sta} between 0.5 and 2.5. Staticization is performed between 50k and 75k iterations, resulting in 9%–43% of ST-Anchors being converted into static representations. Scenes with weaker motion or larger near-static regions use a relatively larger staticization threshold, while scenes with more significant motion use a more conservative threshold to avoid prematurely removing necessary dynamic degrees of freedom.

III. MORE RESULTS

TABLE II
 QUANTITATIVE RESULTS ON THE N3DV DATASET. LPIPS IS EVALUATED USING THE ALEXNET BACKBONE. FOR ANCHOR-BASED METHODS, WE REPORT THE NUMBER OF ANCHORS; FOR GAUSSIAN-BASED METHODS, WE REPORT THE NUMBER OF GAUSSIAN PRIMITIVES.

Scene	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	#Anchors	#Gaussians	Storage (MB) $\downarrow$
coffee_martini	4DGaussian	28.47	0.897	0.105	–	138K	42.04
	ST-GS/50	28.37	0.917	0.067	–	1,357K	231.72
	ST-GS/100	28.42	0.914	0.068	–	1,120K	161.30
	4D-Rotor-GS	28.63	0.913	0.082	–	717K	565.08
	ADC-GS	26.50	0.880	0.126	14K	–	11.27
	Anchored-4DGS	28.83	0.912	0.090	342K	–	154.96
	4D-Scaffold-GS	28.95	0.918	0.072	378K	–	117.64
	Ours	28.86	0.924	0.066	55K	–	39.49
cook_spinach	4DGaussian	31.71	0.924	0.077	–	125K	40.07
	ST-GS/50	32.70	0.956	0.036	–	720K	154.49
	ST-GS/100	32.96	0.950	0.037	–	834K	121.81
	4D-Rotor-GS	31.83	0.943	0.052	–	541K	434.17
	ADC-GS	30.87	0.932	0.084	8K	–	4.42
	Anchored-4DGS	31.99	0.941	0.062	263K	–	129.38
	4D-Scaffold-GS	32.25	0.947	0.043	502K	–	180.92
	Ours	32.86	0.956	0.040	60K	–	51.01
cut_roasted_beef	4DGaussian	32.29	0.932	0.070	–	127K	39.98
	ST-GS/50	33.41	0.958	0.036	–	682K	148.71
	ST-GS/100	32.97	0.954	0.037	–	735K	106.22
	4D-Rotor-GS	32.44	0.948	0.047	–	548K	439.79
	ADC-GS	31.37	0.933	0.087	8K	–	5.78
	Anchored-4DGS	32.55	0.948	0.053	326K	–	146.71
	4D-Scaffold-GS	32.88	0.954	0.036	514K	–	185.54
	Ours	33.20	0.956	0.038	67K	–	45.39
flame_salmon	4DGaussian	28.91	0.904	0.099	–	367K	99.21
	ST-GS/50	29.18	0.924	0.063	–	1,562K	258.63
	ST-GS/100	28.68	0.915	0.064	–	1,238K	180.90
	4D-Rotor-GS	29.05	0.922	0.074	–	633K	502.07
	ADC-GS	26.69	0.886	0.125	15K	–	10.05
	Anchored-4DGS	29.26	0.919	0.083	355K	–	159.11
	4D-Scaffold-GS	29.53	0.925	0.063	561K	–	209.03
	Ours	30.09	0.934	0.058	55K	–	40.70
flame_steak	4DGaussian	32.42	0.935	0.067	–	118K	38.34
	ST-GS/50	33.36	0.964	0.029	–	794K	162.84
	ST-GS/100	33.52	0.960	0.031	–	777K	112.88
	4D-Rotor-GS	32.60	0.952	0.041	–	520K	416.56
	ADC-GS	30.86	0.940	0.073	8K	–	4.52
	Anchored-4DGS	32.82	0.951	0.049	288K	–	137.05
	4D-Scaffold-GS	33.07	0.958	0.033	409K	–	134.48
	Ours	33.40	0.961	0.035	49K	–	40.89
sear_steak	4DGaussian	32.50	0.934	0.068	–	119K	32.80
	ST-GS/50	34.02	0.963	0.027	–	789K	166.09
	ST-GS/100	33.17	0.959	0.033	–	776K	111.23
	4D-Rotor-GS	32.71	0.956	0.040	–	498K	398.37
	ADC-GS	30.77	0.937	0.069	7K	–	8.48
	Anchored-4DGS	32.71	0.951	0.047	222K	–	113.39
	4D-Scaffold-GS	33.10	0.962	0.029	305K	–	79.14
	Ours	34.01	0.963	0.033	59K	–	49.67

TABLE III

QUANTITATIVE RESULTS ON THE GOOGLE IMMERSIVE DATASET. LPIPS IS EVALUATED USING THE ALEXNET BACKBONE. FOR ANCHOR-BASED METHODS, WE REPORT THE NUMBER OF ANCHORS; FOR GAUSSIAN-BASED METHODS, WE REPORT THE NUMBER OF GAUSSIAN PRIMITIVES.

Scene	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	#Anchors	#Gaussians	Storage (MB) $\downarrow$
02_Flames	4DGaussian	25.63	0.912	0.099	–	367K	105.70
	ST-GS/50	30.18	0.935	0.068	–	1,393K	170.23
	ST-GS/100	29.68	0.928	0.080	–	1,231K	150.36
	4D-Rotor-GS	29.25	0.927	0.132	–	362K	292.00
	ADC-GS	22.93	0.814	0.323	8K	–	7.29
	Anchored-4DGS	29.01	0.911	0.097	326K	–	145.27
	4D-Scaffold-GS	29.44	0.932	0.075	542K	–	183.91
	Ours	29.89	0.939	0.073	96K	–	79.83
09_Alexa_Meade_Exhibit	4DGaussian	24.95	0.893	0.098	–	368K	106.00
	ST-GS/50	26.42	0.885	0.109	–	2,543K	310.67
	ST-GS/100	26.11	0.876	0.121	–	2,008K	245.18
	4D-Rotor-GS	24.05	0.886	0.159	–	362K	292.51
	ADC-GS	23.07	0.780	0.387	12K	–	10.56
	Anchored-4DGS	25.97	0.862	0.136	266K	–	118.64
	4D-Scaffold-GS	26.92	0.884	0.112	428K	–	145.27
	Ours	26.81	0.895	0.095	59K	–	49.72
10_Alexa	4DGaussian	26.03	0.946	0.076	–	368K	105.82
	ST-GS/50	29.07	0.917	0.081	–	1,682K	205.44
	ST-GS/100	28.61	0.909	0.093	–	1,453K	177.42
	4D-Rotor-GS	26.50	0.921	0.194	–	312K	252.98
	ADC-GS	24.18	0.846	0.482	7K	–	6.12
	Anchored-4DGS	28.40	0.896	0.112	298K	–	132.89
	4D-Scaffold-GS	28.60	0.917	0.086	658K	–	223.64
	Ours	28.43	0.941	0.090	72K	–	60.41
12_Cave	4DGaussian	24.71	0.871	0.136	–	361K	104.16
	ST-GS/50	26.45	0.887	0.107	–	2,358K	288.06
	ST-GS/100	26.19	0.879	0.118	–	2,030K	247.84
	4D-Rotor-GS	26.79	0.884	0.225	–	222K	183.03
	ADC-GS	22.81	0.793	0.312	29K	–	25.80
	Anchored-4DGS	25.49	0.860	0.147	246K	–	109.60
	4D-Scaffold-GS	26.76	0.883	0.120	474K	–	161.02
	Ours	27.71	0.893	0.126	96K	–	82.12

TABLE IV

QUANTITATIVE RESULTS ON THE ENERF-OUTDOOR DATASET. LPIPS IS EVALUATED USING THE ALEXNET BACKBONE. FOR ANCHOR-BASED METHODS, WE REPORT THE NUMBER OF ANCHORS; FOR GAUSSIAN-BASED METHODS, WE REPORT THE NUMBER OF GAUSSIAN PRIMITIVES.

Scene	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	#Anchors	#Gaussians	Storage (MB) $\downarrow$
actor1_4	4DGaussian	23.97	0.731	0.206	–	193K	60.22
	ST-GS/50	25.62	0.780	0.188	–	646K	98.21
	ST-GS/100	25.16	0.781	0.186	–	320K	48.69
	4D-Rotor-GS	24.66	0.646	0.508	–	472K	378.01
	ADC-GS	21.60	0.499	0.477	19K	–	13.37
	Anchored-4DGS	27.22	0.772	0.199	189K	–	88.79
	4D-Scaffold-GS	26.16	0.805	0.102	218K	–	77.76
	Ours	27.25	0.787	0.140	47K	–	38.60
actor2_3	4DGaussian	24.54	0.702	0.229	–	188K	58.94
	ST-GS/50	25.77	0.778	0.187	–	663K	100.86
	ST-GS/100	25.19	0.771	0.184	–	325K	49.41
	4D-Rotor-GS	24.60	0.659	0.470	–	429K	344.23
	ADC-GS	21.57	0.478	0.534	18K	–	13.13
	Anchored-4DGS	26.54	0.749	0.243	188K	–	88.30
	4D-Scaffold-GS	26.00	0.798	0.112	270K	–	96.21
	Ours	27.28	0.785	0.150	47K	–	38.64

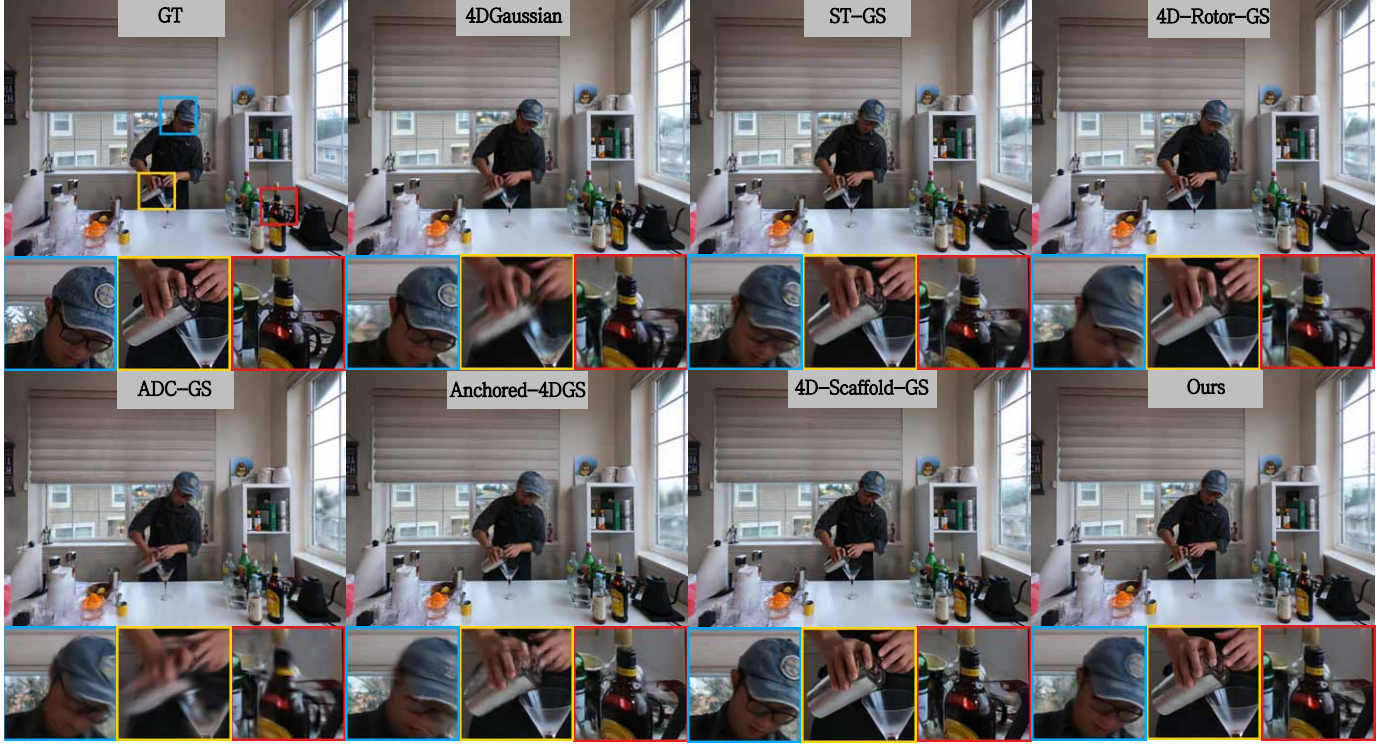


Fig. 1. Qualitative comparison on the *coffee_martini* scene of the N3DV dataset.



Fig. 2. Qualitative comparison on the *cook_spinach* scene of the N3DV dataset.

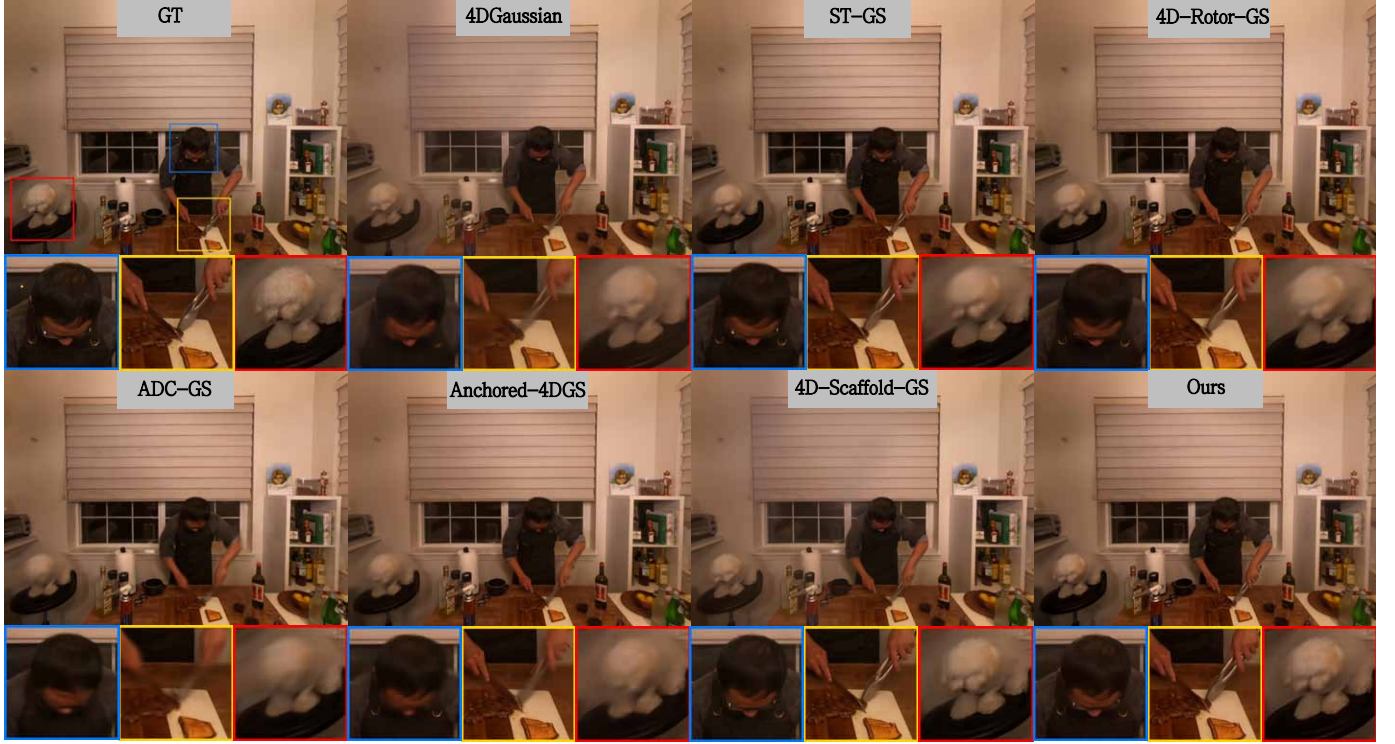


Fig. 3. Qualitative comparison on the *cut_roasted_beef* scene of the N3DV dataset.

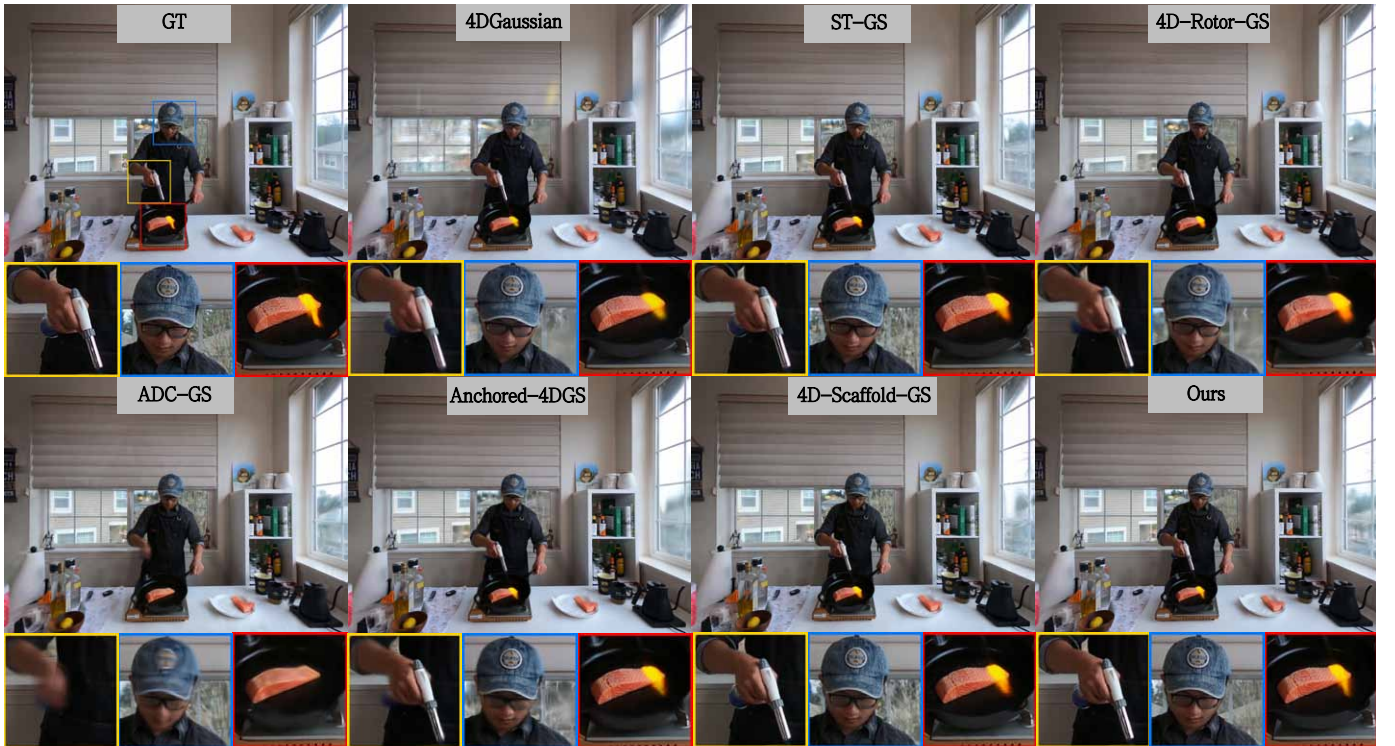


Fig. 4. Qualitative comparison on the *flame_salmon* scene of the N3DV dataset.



Fig. 5. Qualitative comparison on the *flame_steak* scene of the N3DV dataset.



Fig. 6. Qualitative comparison on the *sear_steak* scene of the N3DV dataset.

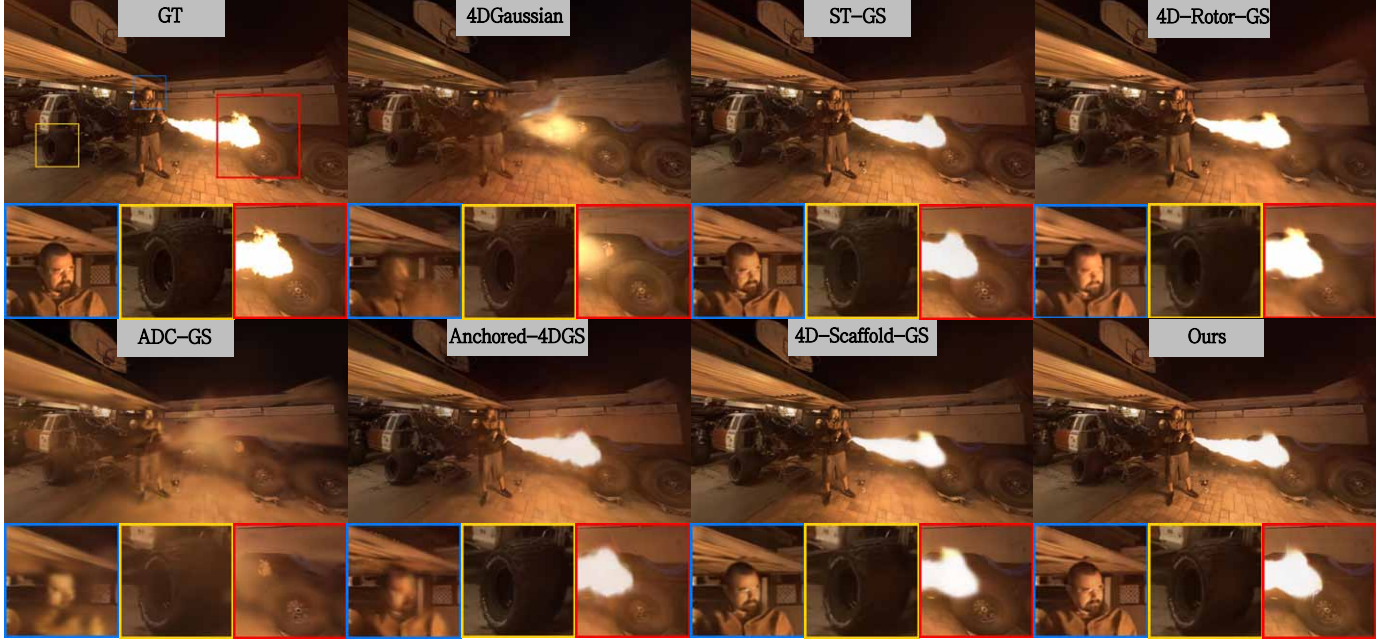


Fig. 7. Qualitative comparison on the *02_Flames* scene of the Google Immersive dataset.

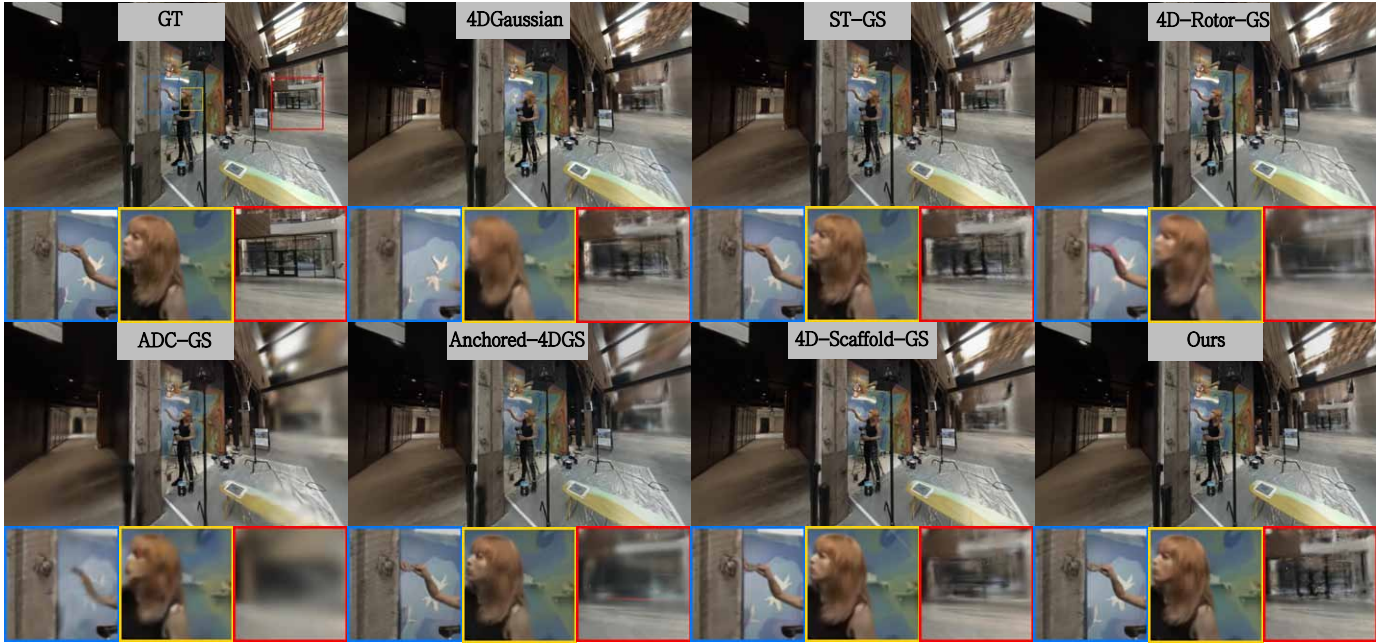


Fig. 8. Qualitative comparison on the *09_Alexa_Meade_Exhibit* scene of the Google Immersive dataset.

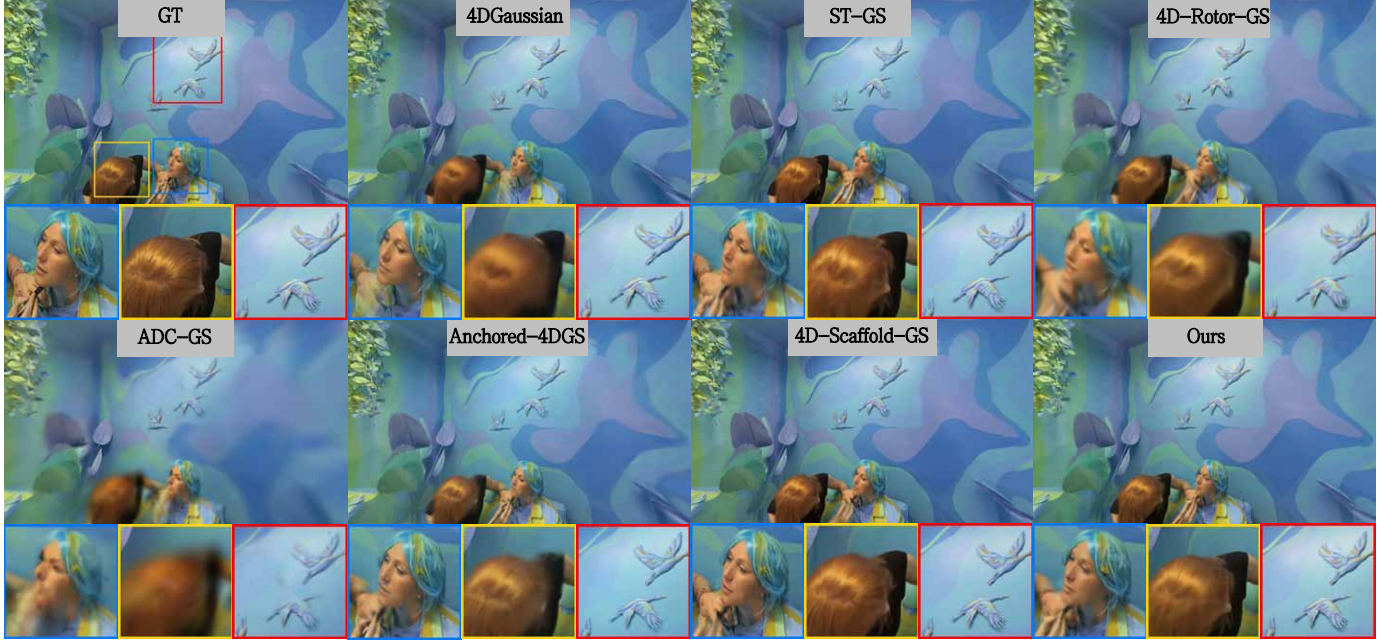


Fig. 9. Qualitative comparison on the *10_Alexa* scene of the Google Immersive dataset.

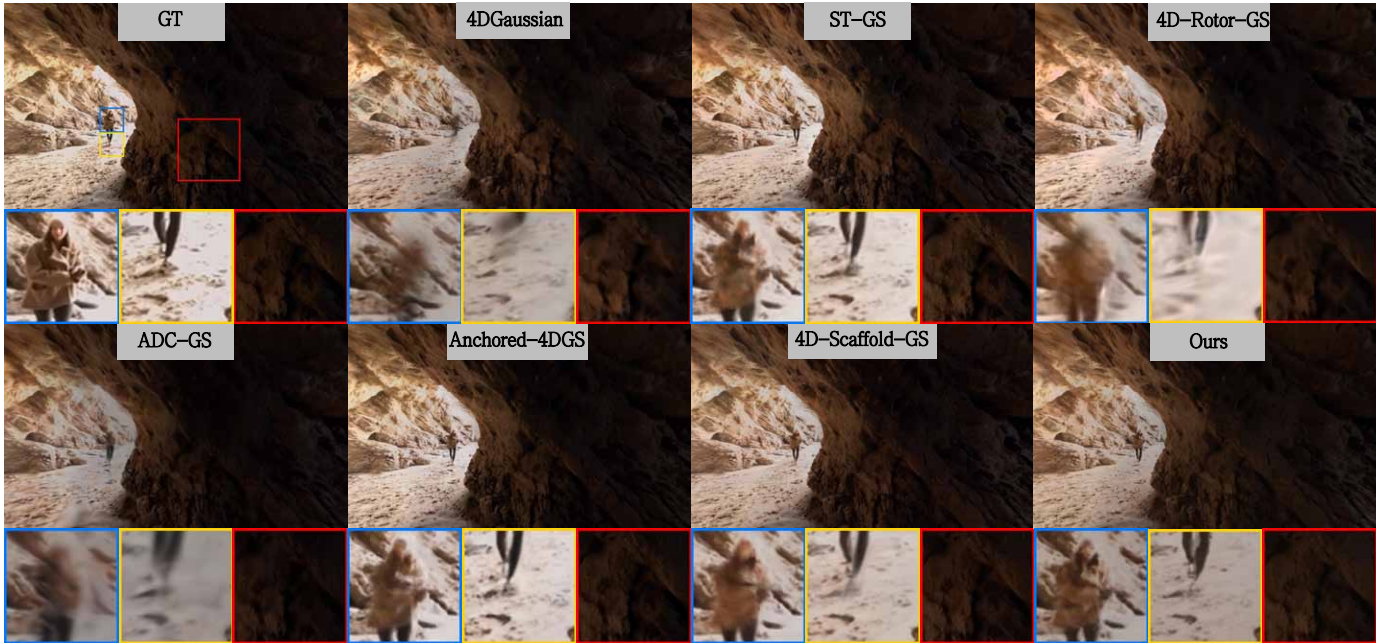


Fig. 10. Qualitative comparison on the *12_Cave* scene of the Google Immersive dataset.

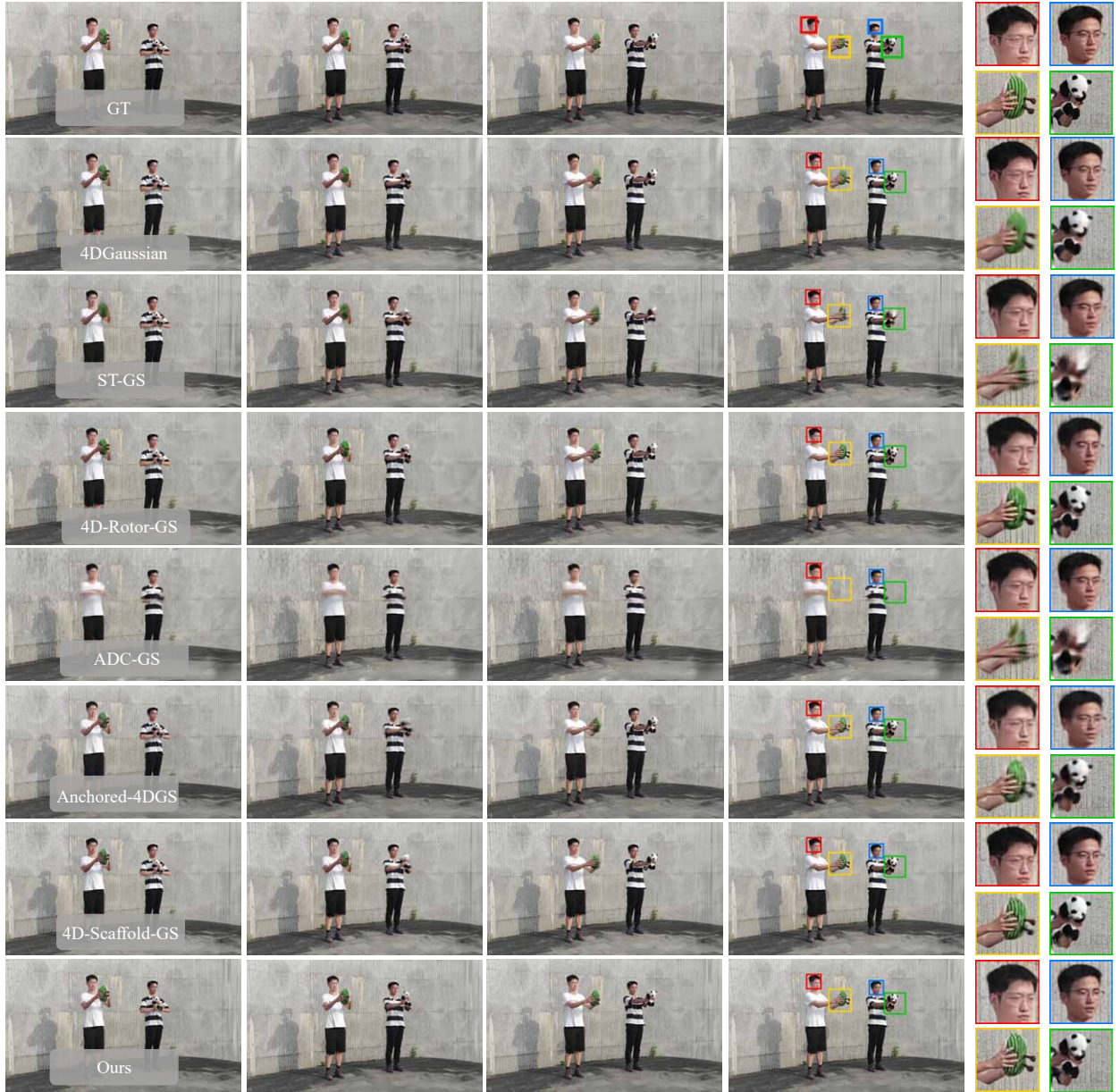


Fig. 11. Qualitative comparison on the *actor1_4* scene of the ENeRF-Outdoor dataset.

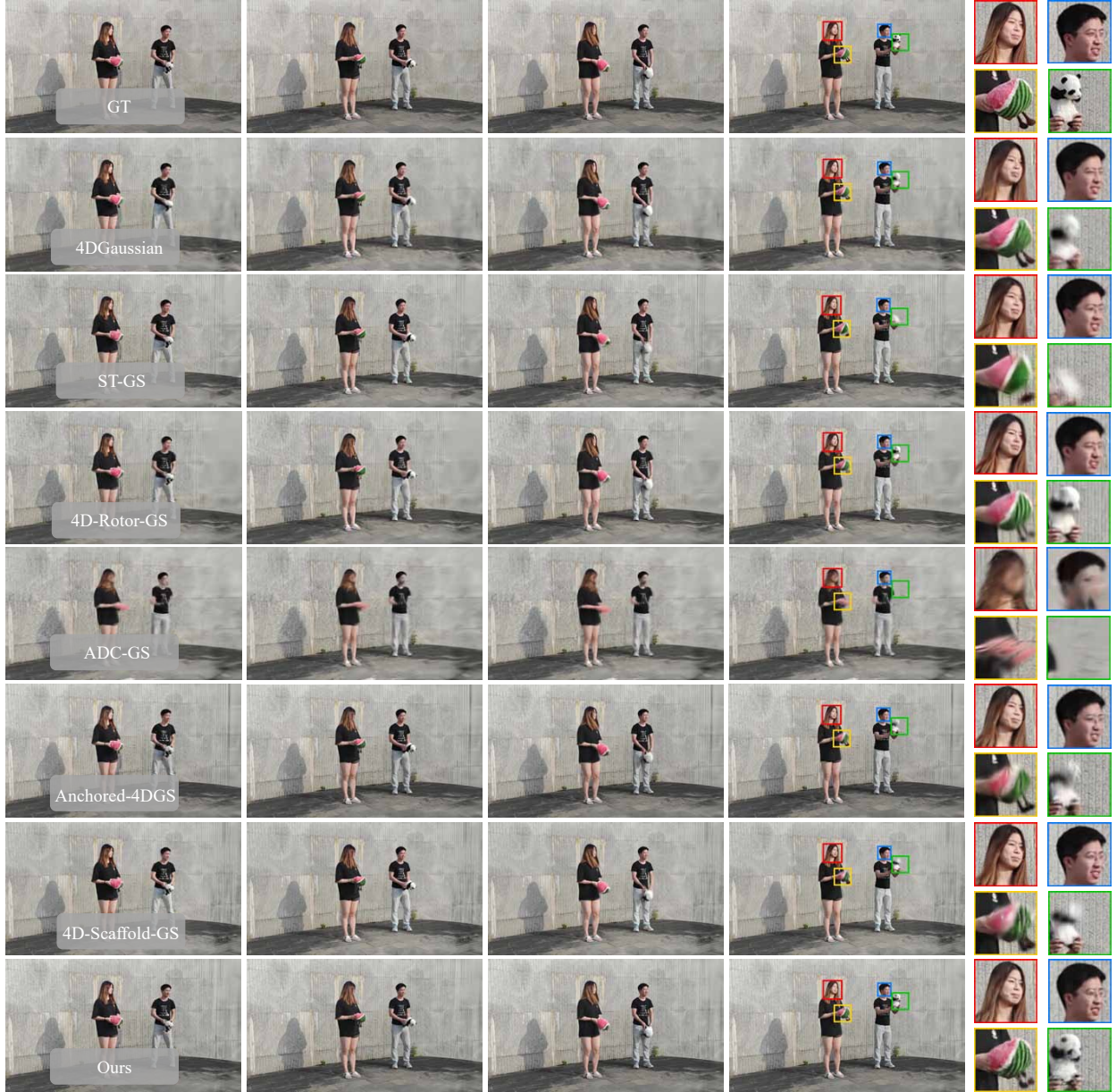


Fig. 12. Qualitative comparison on the *actor2_3* scene of the ENeRF-Outdoor dataset.